

Fəsil 16. RNN-lər və Attention ilə Təbii Dil Emalı

Alan Turing 1950-ci ildə məşhur Turing testini təsəvvür edərkən, maşının insan zəkasına uyğunlaşma qabiliyyətini qiymətləndirmək üçün bir üsul təklif etdi. O, şəkillərdə pişikləri tanımaq, şahmat oynamaq, musiqi bəstələmək və ya labirintdən çıxmaq kimi bir çox şeyi sınaqdan keçirə bilərdi, lakin maraqlıdır ki, o, dilçilik tapşırığını seçdi. Daha dəqiq desək, o, həmsöhbətini insan olduğuna inandıra bilən bir *chatbot* yaratdı. Bu testin zəif tərəfləri var: sərt kodlaşdırılmış qaydalar dərrakəsiz və ya sadələvh insanları aldada bilər (məsələn, maşın bəzi açarlara cavab olaraq qeyri-müəyyən əvvəlcədən təyin edilmiş cavablar verə bilər, ən qərribə cavablarına bəhanə üçün zarafat etdiyini və ya sərxoş olduğunu iddia edə bilər, ya da çətin suallara öz sualları ilə cavab verərək onlardan yayına bilər) və insan zəkasının bir çox aspektləri tamamilə nəzərə alınmır (məsələn, üz ifadələri kimi qeyri-şifahi ünsiyyəti şərh etmək və ya əl ilə tapşırığı öyrənmək qabiliyyəti). Lakin test, dilə yiyələnməyin, şübhəsiz ki, *Homo sapiens* -in ən böyük idrak qabiliyyəti olduğunu vurğulayır.

Biz yazılı və şifahi dili mənimsəyən bir maşın qura bilərikmi? Bu, NLP tədqiqatının son məqsədidir, lakin bu, bir az genişdir, buna görə praktikada tədqiqatçılar mətn təsnifatı, tərcümə, xülasə, sual-cavab və daha çoxu kimi daha spesifik tapşırıqlara diqqət yetirirlər.

Təbii dil tapşırıqları üçün ümumi bir yanaşma recurrent neural networks (RNNs) istifadə etməkdir. Buna görə də, biz Fəsil 15-də təqdim edilən RNN-ləri araşdırmağa davam edəcəyik, *character RNN* və ya *char-RNN* ilə başlayaraq, bu, cümlədəki növbəti simvolu proqnozlaşdırmaq üçün öyrədilir. Bu, bizə orijinal mətn yaratmağa imkan verəcək. Biz əvvəlcə *stateless RNN* (hər iterasiyada mətnin təsadüfi hissələri üzərində, qalan mətn haqqında heç bir məlumat olmadan öyrənən) istifadə edəcəyik, sonra isə *stateful RNN* (təlim iterasiyaları arasında gizli vəziyyəti qoruyan və qaldığı yerdən oxumağa davam edən, uzun naxışları öyrənməyə imkan verən) quracağıq. Sonra, sentiment analysis (məsələn, film rəylərini oxumaq və rəy verənin film haqqında hisslərini çıxarmaq) üçün bir RNN quracağıq, bu dəfə cümlələri simvollarıdan daha çox sözlər ardıcılığı kimi qəbul edəcəyik. Daha sonra, RNN-lərin ingilis dilini ispan dilinə tərcümə edən neural machine translation (NMT) üçün encoder-decoder arxitekturasını qurmaq üçün necə istifadə oluna biləcəyini göstərəcəyik.

Bu fəslin ikinci hissəsində *attention mechanisms* (diqqət mexanizmləri) araşdıracağıq. Adlarından göründüyü kimi, bunlar modelin qalan hissəsinin hər bir zaman addımında diqqət yetirməli olduğu giriş hissəsini seçməyi öyrənən neyron şəbəkə komponentləridir. Əvvəlcə, attention istifadə edərək RNN əsaslı encoder-decoder arxitekturasının performansını artıracağıq. Sonra, RNN-ləri tamamilə tərk edəcəyik və yalnız attention əsaslı çox uğurlu bir arxitektura olan *transformer* -dan istifadə edərək tərcümə modeli quracağıq. Daha sonra, son bir neçə

İldə NLP-dəki ən mühüm irəliləyişlərdən bəzilərini, o cümlədən GPT və BERT kimi inanılmaz dərəcədə güclü dil modellərini müzakirə edəcəyik, hər ikisi transformer-lara əsaslanır. Nəhayət, Hugging Face-in əla Transformers kitabxanası ilə necə başlamaq lazım olduğunu göstərəcəyəm. Şekspir kimi yazı bilən sadə və əyləncəli bir model ilə başlayaq.

Simvol RNN istifadə edərək Şekspir Mətni Yaradılması

Andrej Karpathy-nin 2015-ci ildəki məşhur "The Unreasonable Effectiveness of Recurrent Neural Networks" blog yazısında, bir RNN-nin cümlədəki növbəti simvolu proqnozlaşdırmaq üçün necə öyrədildiyini göstərdi. Bu *char-RNN* daha sonra bir-bir simvol yaradaraq yeni mətn yaratmaq üçün istifadə edilə bilər. Budur, Şekspirin bütün əsərləri üzərində öyrədilmiş bir char-RNN modeli tərəfindən yaradılan mətnin kiçik bir nümunəsi:

PANDARUS:

*Alas, I think he shall be come approached and the day
When little strain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.*

Tam bir şah əsər deyil, lakin modelin yalnız bir cümlədəki növbəti simvolu proqnozlaşdırmağı öyrənməklə sözləri, qrammatikanı, durğu işarələrini və daha çoxunu öyrənmə bilməsi hələ də təsir edicidir. Bu, bizim *language model* (dil modeli) üçün ilk nümunəmizdir; bu fəslin sonrakı hissələrində müzakirə ediləcək oxşar (lakin daha güclü) dil modelləri müasir NLP-nin əsasını təşkil edir. Bu bölmənin qalan hissəsində biz char-RNN-ni addım-addım quracağıq, dataset-in yaradılması ilə başlayacağıq.

Təlim Dataset-inin Yaradılması

Əvvəlcə Keras-ın rahat `tf.keras.utils.get_file()` funksiyasından istifadə edərək Şekspirin bütün əsərlərini yükləyək. Məlumat Andrej Karpathy-nin char-rnn layihəsindən yüklənir:

```
import tensorflow as tf

shakespeare_url = "https://hml.info/shakespeare" # shortcut URL
filepath = tf.keras.utils.get_file("shakespeare.txt", shakespeare_url)
with open(filepath) as f:

    shakespeare_text = f.read()
```

İlk bir neçə sətiri çap edək:

```
>>> print(shakespeare_text[:80])
First Citizen: Before we proceed any further, hear me speak.

All: Speak, speak.
```

Şekspir kimi görünür!

Sonra, bu mətni kodlaşdırmaq üçün `tf.keras.layers.TextVectorization` qatından (Fəsil 13-də təqdim edilmişdir) istifadə edəcəyik. `split="character"` təyin edərək simvol səviyyəsində kodlaşdırma əldə edəcəyik (defolt söz səviyyəsindən fərqli olaraq) və `standardize="lower"` istifadə edərək mətni kiçik hərflərə çevirəcəyik (bu, tapşırığı sadələşdirəcək):

```
text_vec_layer = tf.keras.layers.TextVectorization(split="character",
standardize="lower")
text_vec_layer.adapt([shakespeare_text])

encoded = text_vec_layer([shakespeare_text])[0]
```

İndi hər bir simvol 2-dən başlayaraq tam ədədə xəritələşdirilir. `TextVectorization` qatı 0 dəyərini padding token-lər üçün, 1-i isə bilinməyən simvollar üçün ayırmışdır. Biz hələlik bu token-lərə ehtiyacımız yoxdur, buna görə simvol ID-lərindən 2 çıxmaq və fərqli simvolların sayını və ümumi simvolların sayını hesablayaq:

```
encoded -= 2 # drop tokens 0 (pad) and 1 (unknown), which we will not use
n_tokens = text_vec_layer.vocabulary_size() - 2 # number of distinct chars
= 39
```

```
dataset_size = len(encoded) # total number of chars = 1,115,394
```

Sonra, Fəsil 15-də etdiyimiz kimi, bu çox uzun ardıcılığı sequence-to-sequence RNN-ni öyrətmək üçün istifadə edə biləcəyimiz pəncərələr dataset-inə çevirə bilərik. Hədəflər girişlərə bənzəyəcək, lakin bir zaman addımı "gələcəyə" sürüsdürüləcək. Məsələn, dataset-dəki bir nümunə "to be or not to b" (son "e" olmadan) mətnini təmsil edən simvol ID-lərinin ardıcılığı ola bilər və müvafiq hədəf – "o be or not to be" (son "e" ilə, lakin başlanğıc "t" olmadan) mətnini təmsil edən simvol ID-lərinin ardıcılığı. Uzun bir simvol ID ardıcılığını giriş/hədəf pəncərə cütləri dataset-inə çevirmək üçün kiçik bir yardımçı funksiya yazaq:

```
def to_dataset(sequence, length, shuffle=False, seed=None, batch_size=32):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=1, drop_remainder=True)
    ds = ds.flat_map(lambda window_ds: window_ds.batch(length + 1))
```

```

if shuffle:
    ds = ds.shuffle(buffer_size=100_000, seed=seed)
    ds = ds.batch(batch_size)

    return ds.map(lambda window: (window[:, :-1], window[:,
1:])).prefetch(1)

```

Bu funksiya Fəsil 15-də yaratdığımız `to_windows()` xüsusi yardımçı funksiyaşına çox bənzəyir:

- O, giriş kimi bir ardıcılıq alır (yəni kodlaşdırılmış mətn) və bütün istədiyiniz uzunluqdakı pəncərələri ehtiva edən bir dataset yaradır.
- O, uzunluğu bir artırır, çünki hədəf üçün növbəti simvola ehtiyacımız var.
- Sonra, o, pəncərələri (istəyə bağlı olaraq) qarışdırır, onları batch-lərə yığır, giriş/çıxış cütlərinə bölür və prefetching-i aktivləşdirir.

Şəkil 16-1 dataset hazırlıq addımlarını xülasə edir: 11 uzunluğunda pəncərələr və 3 batch ölçüsü göstərilir. Hər bir pəncərənin başlanğıc indeksi yanında göstərilmişdir.

Şəkil 16-1. Qarışdırılmış pəncərələrdən ibarət dataset-in hazırlanması

İndi biz təlim seti, validasiya seti və test seti yaratmağa hazırıq. Mətnin təxminən 90%-ni təlim, 5%-ni validasiya və 5%-ni test üçün istifadə edəcəyik:

```

length = 100
tf.random.set_seed(42)
train_set = to_dataset(encoded[:, 1_000_000], length=length, shuffle=True,
seed=42)
valid_set = to_dataset(encoded[1_000_000:1_060_000], length=length)

test_set = to_dataset(encoded[1_060_000:], length=length)

```

İPUCU

Pəncərə uzunluğunu 100 təyin etdik, lakin onu tənzimləməyə cəhd edə bilərsiniz: daha qısa giriş ardıcılıqlarında RNN-ləri öyrətmək daha asan və sürətlidir, lakin RNN uzunluqdan daha uzun heç bir naxış öyrənə bilməyəcək, buna görə onu çox kiçik etməyin.

Hamısı budur! Dataset-i hazırlamaq ən çətin hissə idi. İndi modeli yaradaq.

Char-RNN Modelinin Qurulması və Təlimi

Dataset-imiz kifayət qədər böyük olduğundan və dil modelləşdirməsi olduqca çətin bir iş olduğundan, sadə bir neçə recurrent neyronlu RNN-dən daha çoxuna ehtiyacımız var. 128 vahiddən ibarət bir GRU qatı ilə model quraq və öyrədək (lazım olduqda qatların və vahidlərin sayını sonra tənzimləyə bilərsiniz):

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16),
    tf.keras.layers.GRU(128, return_sequences=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
metrics=["accuracy"])
model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
    "my_shakespeare_model", monitor="val_accuracy", save_best_only=True)

history = model.fit(train_set, validation_data=valid_set, epochs=10,
callbacks=[model_checkpoint])
```

Bu kodu nəzərdən keçirək:

- İlk qat kimi Embedding qatından istifadə edirik, simvol ID-lərini kodlaşdırmaq üçün (embeddings Fəsil 13-də təqdim edilmişdir). Embedding qatının giriş ölçülərinin sayı fərqli simvol ID-lərinin sayıdır və çıxış ölçülərinin sayı tənzimləyə biləcəyiniz bir hiperparametrdir – indilik onu 16 təyin edək. Embedding qatının girişləri [batch size, window length] formasında 2D tensorlar olacaq, çıxışı isə [batch size, window length, embedding size] formasında 3D tensor olacaq.
- Çıxış qatı üçün Dense qatından istifadə edirik: onun 39 vahidi (n_tokens) olmalıdır, çünki mətndə 39 fərqli simvol var və biz hər bir mümkün simvol üçün (hər zaman addımında) ehtimal çıxarmaq istəyirik. 39 çıxış ehtimalı hər zaman addımında 1-ə qədər toplanmalıdır, buna görə Dense qatının çıxışlarına softmax aktivasiya funksiyasını tətbiq edirik.
- Nəhayət, biz bu modeli "sparse_categorical_crossentropy" loss və Nadam optimizatoru ilə kompilyasiya edirik və modeli bir neçə dövr üçün öyrədirik, ModelCheckpoint callback-indən istifadə edərək təlim irəlilədikcə ən yaxşı modeli (validasiya dəqiqliyi baxımından) saxlayırıq.

İPUCU

Bu kodu GPU aktivləşdirilmiş Colab-da işlədirsinizsə, təlim təxminən bir-iki saat çəkəcək. Bu qədər gözləmək istəmirsinizsə, dövrlərin sayını azalda bilərsiniz, lakin təbii ki, modelin dəqiqliyi yəqin ki, daha aşağı olacaq. Colab sessiyası vaxtı bitərsə,

tez bir zamanda yenidən qoşulduğunuzdan əmin olun, əks halda Colab runtime məhv olacaq.

Bu model mətn preprocessing-i idarə etmir, buna görə onu `tf.keras.layers.TextVectorization` qatını ilk qat kimi və `tf.keras.layers.Lambda` qatını simvol ID-lərindən 2 çıxarmaq üçün (hazırda padding və unknown token-lərindən istifadə etmədiyimiz üçün) ehtiva edən son bir modelə sarırıq:

```
shakespeare_model = tf.keras.Sequential([
    text_vec_layer,
    tf.keras.layers.Lambda(lambda X: X - 2), # no <PAD> or <UNK> tokens
    model
])
```

İndi onu bir cümlədəki növbəti simvolu proqnozlaşdırmaq üçün istifadə edək:

```
>>> y_proba = shakespeare_model.predict(["To be or not to b"])[0, -1]
>>> y_pred = tf.argmax(y_proba) # choose the most probable character ID
>>> text_vec_layer.get_vocabulary()[y_pred + 2]
```

```
'e'
```

Əla, model növbəti simvolu düzgün proqnozlaşdırdı. İndi bu modeldən istifadə edərək Şekspir kimi davranma bilərik!

Saxta Şekspir Mətni Yaratmaq

Char-RNN modelindən istifadə edərək yeni mətn yaratmaq üçün, ona bir mətn verə bilərik, modelin ən çox ehtimal olunan növbəti hərfi proqnozlaşdırmasını təmin edə bilərik, onu mətnin sonuna əlavə edə bilərik, sonra genişləndirilmiş mətni modelə verərək növbəti hərfi təxmin edə bilərik və s. Buna *greedy decoding* deyilir. Lakin praktikada bu, tez-tez eyni sözlərin təkrar-təkrar təkrarlanmasına səbəb olur. Bunun əvəzinə, növbəti simvolu təxmin edilən ehtimalla bərabər ehtimalla təsadüfi seçə bilərik, TensorFlow-un `tf.random.categorical()` funksiyasından istifadə edərək. Bu, daha müxtəlif və maraqlı mətn yaradacaq. `categorical()` funksiyası sinif log ehtimalları (logits) verildikdə təsadüfi sinif indeksləri nümunə götürür. Məsələn:

```
>>> log_probabilities = tf.math.log([[0.5, 0.4, 0.1]]) # probabilities = 50%, 40%, and 10%
>>> tf.random.set_seed(42)
>>> tf.random.categorical(log_probabilities, num_samples=8) # draw 8 samples
```

```
<tf.Tensor: shape=(1, 8), dtype=int64, numpy=array([[0, 1, 0, 2, 1, 0, 0, 1]])>
```

Yaradılan mətnin müxtəlifliyi üzərində daha çox nəzarətə sahib olmaq üçün, logits-ləri *temperature* (temperatur) adlanan bir ədədə bölə bilərik, onu istədiyimiz kimi tənzimləyə bilərik. Sıfıra yaxın bir temperatur yüksək ehtimal olunan simvolları üstün tutur, yüksək temperatur isə bütün simvollarla bərabər ehtimal verir. Aşağı temperaturlar adətən kifayət qədər sərt və dəqiq mətn (məsələn, riyazi tənliklər) yaradarkən üstünlük təşkil edir, yüksək temperaturlar isə daha müxtəlif və yaradıcı mətn yaradarkən üstünlük təşkil edir. Aşağıdakı `next_char()` xüsusi yardımçı funksiyası giriş mətninə əlavə ediləcək növbəti simvolu seçmək üçün bu yanaşmadan istifadə edir:

```
def next_char(text, temperature=1):
    y_proba = shakespeare_model.predict([text])[0, -1:]
    rescaled_logits = tf.math.log(y_proba) / temperature
    char_id = tf.random.categorical(rescaled_logits, num_samples=1)[0, 0]

    return text_vec_layer.get_vocabulary()[char_id + 2]
```

Sonra, növbəti simvolu almaq üçün `next_char()` funksiyasını təkrar çağıracaq və verilmiş mətnə əlavə edəcək başqa bir kiçik yardımçı funksiya yazı bilərik:

```
def extend_text(text, n_chars=50, temperature=1):
    for _ in range(n_chars):
        text += next_char(text, temperature)

    return text
```

İndi bəzi mətnlər yaratmağa hazırıq! Fərqli temperatur dəyərləri ilə sınayaq:

```
>>> tf.random.set_seed(42)
>>> print(extend_text("To be or not to be", temperature=0.01))
To be or not to be the duke as it is a proper strange death, and the
>>> print(extend_text("To be or not to be", temperature=1))
To be or not to behold?
second push: gremio, lord all, a sistermen,
>>> print(extend_text("To be or not to be", temperature=100))

To be or not to bef ,mt'&o3fpadm!$ wh!nse?bws3est--vgerdjwt?c-y-ewznq
```

Şekspir istidən əziyyət çəkirmiş kimi görünür. Daha inandırıcı mətn yaratmaq üçün ümumi bir texnika, yalnız ən yaxşı k simvoldan və ya ümumi ehtimalı müəyyən bir həddi aşan ən kiçik ən yaxşı simvollar dəstindən nümunə götürməkdir (buna *nucleus*

sampling deyilir). Alternativ olaraq, *beam search* (bu fəsildə sonra müzakirə edəcəyik) və ya daha çox GRU qatı və qat başına daha çox neyron istifadə etmək, daha uzun müddət təlim etmək və lazım olduqda bir az regularizasiya əlavə etmək olar. Həmçinin qeyd edək ki, model hazırda yalnız 100 simvol olan uzunluqdan daha uzun naxışları öyrənə bilmir. Bu pəncərəni daha böyük etməyə cəhd edə bilərsiniz, lakin bu, təlimi daha çətinləşdirəcək və hətta LSTM və GRU hüceyrələri çox uzun ardıcılıqları idarə edə bilməz. Alternativ bir yanaşma stateful RNN istifadə etməkdir.

Stateful RNN

İndiyə qədər biz yalnız *stateless RNN* -lərdən istifadə etmişik: hər bir təlim iterasiyasında model sıfırlarla dolu gizli vəziyyətlə başlayır, sonra bu vəziyyəti hər zaman addımında yeniləyir və son zaman addımından sonra artıq lazım olmadığı üçün onu atır. Əgər RNN-yə təlim batch-i emal etdikdən sonra bu son vəziyyəti qorumağı və onu növbəti təlim batch-i üçün ilkin vəziyyət kimi istifadə etməyi tapsaydıq? Bu yolla model yalnız qısa ardıcılıqlar vasitəsilə backpropagation etsə də, uzunmüddətli naxışları öyrənə bilərdi. Buna *stateful RNN* deyilir. Gəlin onu necə quracağımızı nəzərdən keçirək.

Birincisi, qeyd edək ki, stateful RNN yalnız o zaman məna kəsb edir ki, bir batch-dəki hər bir giriş ardıcılığı əvvəlki batch-dəki müvafiq ardıcılığın dayandığı yerdən dəqiq başlasın. Beləliklə, stateful RNN qurmaq üçün ilk etməli olduğumuz şey ardıcıl və üst-üstə düşməyən giriş ardıcılıqlarından istifadə etməkdir (stateless RNN-ləri öyrətmək üçün istifadə etdiyimiz qarışdırılmış və üst-üstə düşən ardıcılıqlar əvəzinə). `tf.data.Dataset` yaradarkən, `window()` metodunda `shift=1` əvəzinə `shift=length` istifadə etməliyik. Üstəlik, `shuffle()` metodunu çağırمامalıyıq.

Təəssüf ki, stateful RNN üçün dataset hazırlayarkən batching stateless RNN-dən daha çətinidir. Həqiqətən də, əgər `batch(32)` çağırısaq, 32 ardıcıl pəncərə eyni batch-ə qoyulacaq və növbəti batch hər bir pəncərənin dayandığı yerdən davam etməyəcək. Birinci batch 1-32 arasındakı pəncərələri, ikinci batch isə 33-64 arasındakı pəncərələri ehtiva edəcək, buna görə məsələn, hər bir batch-in birinci pəncərəsini (yəni 1 və 33) nəzərə alsaq, onların ardıcıl olmadığını görə bilərsiniz. Bu problemin ən sadə həlli sadəcə 1 batch ölçüsündən istifadə etməkdir. Aşağıdakı `to_dataset_for_stateful_rnn()` xüsusi yardımçı funksiyası stateful RNN üçün dataset hazırlamaq üçün bu strategiyadan istifadə edir:

```
def to_dataset_for_stateful_rnn(sequence, length):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=length, drop_remainder=True)
    ds = ds.flat_map(lambda window: window.batch(length + 1)).batch(1)
```



```

        return ds.map(lambda window: (window[:, :-1], window[:,
1:])).prefetch(1)

stateful_train_set = to_dataset_for_stateful_rnn(encoded[:1_000_000],
length)
stateful_valid_set =
to_dataset_for_stateful_rnn(encoded[1_000_000:1_060_000], length)

stateful_test_set = to_dataset_for_stateful_rnn(encoded[1_060_000:],
length)

```

Şəkil 16-2 bu funksiyanın əsas addımlarını xülasə edir.

Şəkil 16-2. Stateful RNN üçün ardıcıl ardıcılıq fraqmentlərindən ibarət dataset-in hazırlanması

Batching çətinidir, lakin qeyri-mümkün deyil. Məsələn, Şeksprin mətnini 32 bərabər uzunluqlu mətnə bölə, onların hər biri üçün ardıcıl giriş ardıcılıqlarından ibarət bir dataset yarada və nəhayət `tf.data.Dataset.zip(datasets).map(lambda *windows: tf.stack(windows))` istifadə edərək müvafiq ardıcıl batch-lər yarada bilərik, burada bir batch-dəki n ci giriş ardıcılığı əvvəlki batch-dəki n ci giriş ardıcılığının dayandığı yerdən dəqiq başlayır (tam kod üçün notebook-a baxın).

İndi stateful RNN yaradaq. Hər bir recurrent qat yaradarkən `stateful` argumentini `True` təyin etməliyik və çünki stateful RNN batch ölçüsünü bilməlidir (çünki o, batch-dəki hər bir giriş ardıcılığı üçün bir vəziyyət saxlayacaq). Buna görə, ilk qatda `batch_input_shape` argumentini təyin etməliyik. Qeyd edək ki, ikinci ölçünü qeyri-müəyyən qoya bilərik, çünki giriş ardıcılıqları istənilən uzunluqda ola bilər:

```

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16,
                               batch_input_shape=[1, None]),
    tf.keras.layers.GRU(128, return_sequences=True, stateful=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])

```

Hər bir dövrün sonunda, mətnin əvvəlinə qayıtmazdan əvvəl vəziyyətləri sıfırlamalıyıq. Bunun üçün kiçik bir xüsusi Keras callback istifadə edə bilərik:

```

class ResetStatesCallback(tf.keras.callbacks.Callback):
    def on_epoch_begin(self, epoch, logs):

        self.model.reset_states()

```

İndi modeli kompilyasiya edə və callback-imizdən istifadə edərək öyrədə bilərik:

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",  
metrics=["accuracy"])  
history = model.fit(stateful_train_set, validation_data=stateful_valid_set,  
                    epochs=10, callbacks=[ResetStatesCallback(),  
model_ckpt])
```

İPUCU

Bu model öyrədildikdən sonra, onu yalnız təlim zamanı istifadə olunan batch ölçüsü ilə eyni ölçülü batch-lər üçün proqnozlar vermək üçün istifadə etmək mümkün olacaq. Bu məhdudiyyətin qarşısını almaq üçün eyni *stateless* model yaradın və *stateful* modelin çəkirlərini bu modelə köçürün.

Maraqlıdır ki, char-RNN modeli sadəcə növbəti simvolu proqnozlaşdırmaq üçün öyrədilsə də, bu zahirən sadə tapşırıq əslində bəzi daha yüksək səviyyəli tapşırıqları da öyrənməyi tələb edir. Məsələn, "Great movie, I really" dan sonrakı simvolu tapmaq üçün cümlənin müsbət olduğunu anlamaq faydalıdır, buna görə "I" ("loved" üçün) hərfinin "h" ("hated" üçün) hərfindən daha çox ehtimal olması. Əslində, Alec Radford və digər OpenAI tədqiqatçılarının 2017-ci il məqaləsi, müəlliflərin böyük bir dataset üzərində böyük bir char-RNN bənzəri modelini necə öyrətdiklərini və neyronlardan birinin əla sentiment analysis təsnifatçısı kimi çıxış etdiyini tapdıqlarını təsvir edir: model heç bir etiket olmadan öyrədilsə də, *sentiment neuron* – onların adlandırdığı kimi – sentiment analysis benchmark-larında ən müasir performansla çatdı. Bu, NLP-də unsupervised pretraining-in xəbərçisi və motivasiyası oldu.

Lakin biz unsupervised pretraining-i araşdırmazdan əvvəl, diqqətimizi söz səviyyəli modellərə və onların sentiment analysis üçün supervised şəkildə necə istifadə oluna biləcəyinə yönəldək. Bu prosesdə, masking istifadə edərək dəyişən uzunluqlu ardıcılıqları necə idarə edəcəyinizi öyrənəcəksiniz.

Sentiment Analysis

Mətn yaratmaq əyləncəli və öyrədici ola bilər, lakin real layihələrdə NLP-nin ən çox yayılmış tətbiqlərindən biri mətn təsnifatıdır – xüsusilə sentiment analysis. MNIST dataset-də şəkil təsnifatı kompüter görməsinin "Hello world!"-ıdırsa, IMDb rəyləri dataset-i üzərində sentiment analysis təbii dil emalının "Hello world!"-ıdır. IMDb dataset-i məşhur Internet Movie Database-dən çıxarılmış 50,000 ingilis dilində film rəyindən (25,000 təlim, 25,000 test) və hər bir rəy üçün mənfi (0) və ya müsbət (1)

olduğunu göstərən sadə ikili hədəfdən ibarətdir. MNIST kimi, IMDb rəyləri dataset-i yaxşı səbəblərə görə məşhurdur: laptopda ağlabatan müddətdə həll etmək üçün kifayət qədər sadədir, lakin əyləncəli və faydalı olmaq üçün kifayət qədər çətinlik təşkil edir.

Gəlin IMDb dataset-i TensorFlow Datasets kitabxanasından istifadə edərək yükləyək (Fəsil 13-də təqdim edilmişdir). Təlim setinin ilk 90%-ni təlim, qalan 10%-ni isə validasiya üçün istifadə edəcəyik:

```
import tensorflow_datasets as tfds

raw_train_set, raw_valid_set, raw_test_set = tfds.load(
    name="imdb_reviews",
    split=["train[:90%]", "train[90%:]", "test"],
    as_supervised=True)
tf.random.set_seed(42)
train_set = raw_train_set.shuffle(5000, seed=42).batch(32).prefetch(1)
valid_set = raw_valid_set.batch(32).prefetch(1)

test_set = raw_test_set.batch(32).prefetch(1)
```

İPUCU

Keras həmçinin IMDb dataset-i yükləmək üçün bir funksiya daxildir, əgər istəsəniz: `tf.keras.datasets.imdb.load_data()`. Rəylər artıq söz ID-ləri ardıcılığı kimi preprocess edilmişdir.

Gəlin bir neçə rəyə baxaq:

```
>>> for review, label in raw_train_set.take(4):
...     print(review.numpy().decode("utf-8"))
...     print("Label:", label.numpy())
...
This was an absolutely terrible movie. Don't be lured in by Christopher
[...]
Label: 0
I have been known to fall asleep during films, but this is usually due to
[...]
Label: 0
Mann photographs the Alberta Rocky Mountains in a superb fashion, and [...]
Label: 0
This is the kind of film for a snowy Sunday afternoon when the rest of the
[...]
```

Label: 1

Bəzi rəyləri təsnif etmək asandır. Məsələn, birinci rəy "terrible movie" sözlərini ilk cümlədə ehtiva edir. Lakin bir çox halda işlər o qədər də sadə deyil. Məsələn, üçüncü rəy müsbət başlasa da, son nəticədə mənfi rəydir (etiket 0).

Bu tapşırıq üçün bir model qurmaq üçün mətni preprocess etməliyik, lakin bu dəfə onu simvollar əvəzinə sözlərə ayıracağıq. Bunun üçün yenidən `tf.keras.layers.TextVectorization` qatından istifadə edə bilərik. Qeyd edək ki, o, söz hüdudlarını müəyyən etmək üçün boşluqlardan istifadə edir, bu, bəzi dillərdə yaxşı işləməyəcək. Məsələn, Çin yazısı sözlər arasında boşluq istifadə etmir, Vyetnam dili hətta sözlər daxilində boşluqlardan istifadə edir və Alman dili tez-tez boşluq olmadan çoxsaylı sözləri birləşdirir. Hətta ingilis dilində, boşluqlar mətni tokenizə etmək üçün hər zaman ən yaxşı yol deyil: "San Francisco" və ya "#ILoveDeepLearning" düşünün.

Xoşbəxtlikdən, bu məsələləri həll etmək üçün həll yolları var. 2016-cı ildə Edinburq Universitetindən Rico Sennrich və digərlərinin məqaləsində mətni subword səviyyəsində tokenizə və detokenizə etmək üçün bir neçə üsul araşdırıldı. Bu yolla, hətta modeliniz heç vaxt görmədiyi nadir bir sözlə qarşılaşsa belə, onun nə demək olduğunu ağlabatan şəkildə təxmin edə bilər. Məsələn, model heç vaxt "smartest" sözünü görməsə də, "smart" sözünü öyrənibsə və "est" şəkilçisinin "ən" mənasını verdiyini öyrənibsə, "smartest" mənasını çıxara bilər. Müəlliflərin qiymətləndirdiyi texnikalardan biri *byte pair encoding* (BPE)-dir. BPE bütün təlim setini fərdi simvollar (boşluqlar daxil olmaqla) bölməklə işləyir, sonra lüğət istənilən ölçüyə çatana qədər ən tez-tez rast gəlinən qonşu cütləri təkrar birləşdirir.

Google-da Taku Kudo tərəfindən 2018-ci il məqaləsi subword tokenization-i daha da təkmilləşdirdi, çox vaxt tokenization-dən əvvəl dilə xas preprocessing ehtiyacını aradan qaldırdı. Üstəlik, məqalə *subword regularization* adlanan yeni bir regularizasiya texnikası təklif etdi, bu, təlim zamanı tokenization-də bəzi təsadüfilik tətbiq edərək dəqiqlik və möhkəmliyi artırır: məsələn, "New England" "New" + "England", ya da "New" + "Eng" + "land", və ya sadəcə "New England" (tək token) kimi tokenizə edilə bilər. Google-un *SentencePiece* layihəsi açıq qaynaq tətbiqini təqdim edir, Taku Kudo və John Richardson tərəfindən məqalədə təsvir edilmişdir. TensorFlow Text kitabxanası həmçinin müxtəlif tokenization strategiyalarını, o cümlədən WordPiece (BPE-nin variantı) tətbiq edir və nəhayət, Hugging Face-in Tokenizers kitabxanası son dərəcə sürətli tokenizer-lərin geniş spektrini tətbiq edir.

Bununla belə, ingilis dilindəki IMDb tapşırığı üçün token hüdudları üçün boşluqlardan istifadə etmək kifayət qədər yaxşı olmalıdır. Beləliklə, gəlin `TextVectorization` qatı yaradaq və onu təlim setinə uyğunlaşdıraq. Lüğəti 1,000 token ilə məhdudlaşdıracağıq, o cümlədən ən tez-tez rast gəlinən 998 söz üstəgəl padding

token və bilinməyən sözlər üçün token, çünki çox nadir sözlərin bu tapşırıq üçün vacib olması ehtimalı azdır və lüğət ölçüsünü məhdudlaşdırmaq modelin öyrənməli olduğu parametrlərin sayını azaldacaq:

```
vocab_size = 1000
text_vec_layer = tf.keras.layers.TextVectorization(max_tokens=vocab_size)

text_vec_layer.adapt(train_set.map(lambda reviews, labels: reviews))
```

Nəhayət, modeli yarada və öyrədə bilərik:

```
embed_size = 128
tf.random.set_seed(42)
model = tf.keras.Sequential([
    text_vec_layer,
    tf.keras.layers.Embedding(vocab_size, embed_size),
    tf.keras.layers.GRU(128),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(loss="binary_crossentropy", optimizer="nadam",
metrics=["accuracy"])

history = model.fit(train_set, validation_data=valid_set, epochs=2)
```

İlk qat hazırladığımız TextVectorization qatıdır, ardınca söz ID-lərini embeddings-ə çevirəcək Embedding qatı gəlir. Embedding matrisi lüğətdəki hər bir token üçün bir sıra (vocab_size) və hər bir embedding ölçüsü üçün bir sütun (bu nümunədə 128 ölçü istifadə edirik, lakin bu, tənzimləyə biləcəyiniz bir hiperparametrdir) ehtiva etməlidir. Sonra bir GRU qatı və tək neyronlu Dense qatı ilə sigmoid aktivasiya funksiyasından istifadə edirik, çünki bu ikili təsnifat tapşırığıdır: modelin çıxışı rəyin film haqqında müsbət hiss bildirdiyi təxmin edilən ehtimal olacaq. Sonra modeli kompilyasiya edirik və onu əvvəllər hazırladığımız dataset üzərində bir neçə dövr üçün fit edirik (və ya daha yaxşı nəticələr üçün daha uzun təlim edə bilərsiniz).

Təəssüf ki, bu kodu işlətsəniz, ümumiyyətlə modelin heç nə öyrənmədiyini görə bilərsiniz: dəqiqlik 50%-ə yaxın qalır, təsadüfi şansdan yaxşı deyil. Niyə belədir? Rəylər müxtəlif uzunluqlara malikdir, buna görə TextVectorization qatı onları token ID ardıcılıqlarına çevirdikdə, daha qısa ardıcılıqları padding token (ID 0) ilə doldurur ki, batch-dəki ən uzun ardıcılıq qədər uzun olsun. Nəticədə, əksər ardıcılıqlar çoxsaylı padding token-lərlə bitir – tez-tez onlarla, hətta yüzlərlə. Biz GRU qatından istifadə etsək də, SimpleRNN qatından daha yaxşı olsa da, onun qısa müddətli yaddaşı hələ də böyük deyil, buna görə çoxsaylı padding token-lərdən keçdikdə, rəyin nə haqqında olduğunu unudur! Bir həll modelə bərabər uzunluqlu cümlələrdən ibarət batch-lər

verməkdir (bu da təlimi sürətləndirir). Digər bir həll RNN-nin padding token-ləri nəzər almamasını təmin etməkdir. Bu, masking istifadə etməklə edilə bilər.

Masking

Modelin padding token-ləri nəzər almamasını Keras-da təmin etmək sadədir: Embedding qatı yaradarkən sadəcə `mask_zero=True` əlavə edin. Bu o deməkdir ki, padding token-lər (ID 0) bütün aşağı qatlar tərəfindən nəzərə alınmayacaq. Hamısı budur! Əvvəlki modeli bir neçə dövr üçün yenidən öyrətsəniz, validasiya dəqiqliyinin tez bir zamanda 80%-dən çox olduğunu görəcəksiniz.

Bu necə işləyir? Embedding qatı `tf.math.not_equal(inputs, 0)`-a bərabər bir *mask tensor* yaradır: bu, girişlərlə eyni formada Boolean tensordur və token ID-ləri 0 olduğu yerlərdə False, əks halda True-dur. Bu mask tensor daha sonra model tərəfindən avtomatik olaraq növbəti qata ötürülür. Əgər həmin qatın `call()` metodunda `mask` argumenti varsa, o, avtomatik olaraq maskanı alır. Bu, qatın müvafiq zaman addımlarını nəzər almamasına imkan verir. Hər bir qat maskanı fərqli şəkildə idarə edə bilər, lakin ümumiyyətlə onlar sadəcə maskalanmış zaman addımlarını (yəni maskanın False olduğu zaman addımları) nəzər almırlar. Məsələn, bir recurrent qat maskalanmış bir zaman addımı ilə qarşılaşdıqda, sadəcə əvvəlki zaman addımının çıxışını köçürür.

Sonra, əgər qatın `supports_masking` atributu True-dursa, maska avtomatik olaraq növbəti qata ötürülür. Qatların `supports_masking=True` olduğu müddətcə, bu şəkildə ötürülməyə davam edir. Məsələn, bir recurrent qatın `return_sequences=True` olduqda `supports_masking` atributu True-dur, lakin `return_sequences=False` olduqda False-dur, çünki bu halda maskaya artıq ehtiyac yoxdur. Beləliklə, əgər `return_sequences=True` olan bir neçə recurrent qatınız varsa, ardınca `return_sequences=False` olan bir recurrent qatı varsa, maska avtomatik olaraq son recurrent qata qədər ötürüləcək: bu qat maskalanmış addımları nəzər almamaq üçün maskadan istifadə edəcək, lakin maskanı daha da ötürməyəcək. Eynilə, sentiment analysis modelində Embedding qatı yaradarkən `mask_zero=True` təyin etsəniz, GRU qatı avtomatik olaraq maskanı alacaq və istifadə edəcək, lakin `return_sequences=True` təyin edilmədiyi üçün onu daha da ötürməyəcək.

İPUCU

Bəzi qatlar maskanı növbəti qata ötürməzdən əvvəl yeniləməlidir: onlar bunu `compute_mask()` metodunu tətbiq etməklə edirlər, bu metod iki argument qəbul edir:

girişlər və əvvəlki maska. Sonra yenilənmiş maskanı hesablayıb qaytarır. `compute_mask()`-in defolt tətbiqi sadəcə əvvəlki maskanı dəyişmədən qaytarır.

Bir çox Keras qatları masking-i dəstəkləyir: SimpleRNN, GRU, LSTM, Bidirectional, Dense, TimeDistributed, Add və bir neçə digərləri (hamısı `tf.keras.layers` paketində). Lakin convolutional qatlar (Conv1D daxil olmaqla) masking-i dəstəkləmir – onların bunu necə edəcəyi aydın deyil.

Əgər maska çıxışa qədər ötürülsə, o, loss-lara da tətbiq edilir, beləliklə maskalanmış zaman addımları loss-a töhfə verməyəcək (onların loss-u 0 olacaq). Bu, modelin ardıcılıqlar çıxardığını fərz edir, sentiment analysis modelimizdə isə belə deyil.

XƏBƏRDARLIQ

LSTM və GRU qatları Nvidia-nın cuDNN kitabxanasına əsaslanan GPU-lar üçün optimallaşdırılmış tətbiqə malikdir. Lakin bu tətbiq masking-i yalnız bütün padding token-ləri ardıcılıqların sonunda olduqda dəstəkləyir. O, həmçinin bir neçə hiperparametr üçün defolt dəyərlərdən istifadə etməyi tələb edir: `activation`, `recurrent_activation`, `recurrent_dropout`, `unroll`, `use_bias` və `reset_after`. Əgər belə deyilsə, bu qatlar (çox daha yavaş) defolt GPU tətbiqinə qayıdacaq.

Əgər masking dəstəyi ilə öz xüsusi qatınızı tətbiq etmək istəyirsinizsə, `call()` metoduna `mask` argumenti əlavə etməli və təbii ki, metodun maskanı istifadə etməsini təmin etməlisiniz. Əlavə olaraq, əgər maska növbəti qatlara ötürülməlidirsə, konstruktorda `self.supports_masking=True` təyin etməlisiniz. Əgər maska ötürülməzdən əvvəl yenilənməlidirsə, `compute_mask()` metodunu tətbiq etməlisiniz.

Əgər modeliniz Embedding qatı ilə başlamırsa, əvəzində `tf.keras.layers.Masking` qatından istifadə edə bilərsiniz: defolt olaraq, o, maskanı `tf.math.reduce_any(tf.math.not_equal(X, 0), axis=-1)` olaraq təyin edir, yəni son ölçüsün sıfırlarla dolu olduğu zaman addımları sonrakı qatlarda maskalanacaq.

Masking qatlarından və avtomatik maska ötürülməsindən istifadə sadə modellər üçün ən yaxşı işləyir. Daha mürəkkəb modellər üçün, məsələn, Conv1D qatlarını recurrent qatlarla qarışdırmaq lazım olduqda, hər zaman işləməyəcək. Belə hallarda, maskanı açıq şəkildə hesablamalı və onu müvafiq qatlara ötürməlisiniz, ya functional API, ya da subclassing API istifadə edərək. Məsələn, aşağıdakı model əvvəlki modelə bərabərdir, lakin functional API ilə qurulur və masking-i əl ilə idarə edir. O, həmçinin bir az dropout əlavə edir, çünki əvvəlki model bir az overfitting edirdi:

```
inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
token_ids = text_vec_layer(inputs)
```

```

mask = tf.math.not_equal(token_ids, 0)
Z = tf.keras.layers.Embedding(vocab_size, embed_size)(token_ids)
Z = tf.keras.layers.GRU(128, dropout=0.2)(Z, mask=mask)
outputs = tf.keras.layers.Dense(1, activation="sigmoid")(Z)

model = tf.keras.Model(inputs=[inputs], outputs=[outputs])

```

Masking üçün son bir yanaşma modelə ragged tensor-lar verməkdir. Praktikada, TextVectorization qatını yaradarkən `ragged=True` təyin etmək kifayətdir, beləliklə giriş ardıcılıqları ragged tensorlar kimi təmsil olunur:

```

>>> text_vec_layer_ragged = tf.keras.layers.TextVectorization(
...     max_tokens=vocab_size, ragged=True)
...
>>> text_vec_layer_ragged.adapt(train_set.map(lambda reviews, labels:
reviews))
>>> text_vec_layer_ragged(["Great movie!", "This is DiCaprio's best
role."])

<tf.RaggedTensor [[86, 18], [11, 7, 1, 116, 217]]>

```

Bu ragged tensor təsvirini padding token-lərindən istifadə edən adi tensor təsviri ilə müqayisə edin:

```

>>> text_vec_layer(["Great movie!", "This is DiCaprio's best role."])
<tf.Tensor: shape=(2, 5), dtype=int64, numpy=
array([[ 86,  18,   0,   0,   0],

       [ 11,   7,   1, 116, 217]])>

```

Keras-in recurrent qatları ragged tensor-lar üçün daxili dəstəyə malikdir, buna görə başqa heç nə etməyə ehtiyac yoxdur: sadəcə bu TextVectorization qatını modelinizdə istifadə edin. `mask_zero=True` ötürməyə və ya maskanı açıq şəkildə idarə etməyə ehtiyac yoxdur – bunların hamısı sizin üçün tətbiq edilir. Bu rahatdır! Bununla belə, 2022-ci ilin əvvəlinə olan məlumata görə, Keras-da ragged tensor-ların dəstəyi hələ olduqca yenidir, buna görə bəzi kobud kənarlar var. Məsələn, hazırda GPU-da işləyərkən hədəf kimi ragged tensor-lardan istifadə etmək mümkün deyil (lakin bu, bu sətirləri oxuduğunuz vaxta qədər həll oluna bilər).

Hansı masking yanaşmasını seçsəniz, bu modeli bir neçə dövr üçün öyrətdikdən sonra, bir rəyin müsbət olub-olmadığını mühakimə etməkdə olduqca yaxşı olacaq. `tf.keras.callbacks.TensorBoard()` callback-indən istifadə etsəniz, öyrənilən embeddings-ləri TensorBoard-da vizuallaşdırı bilərsiniz: "awesome" və "amazing" kimi sözlərin tədricən embedding fəzasının bir tərəfində, "awful" və "terrible" kimi

sözlərin isə digər tərəfdə yığıldığını görmək valehedicidir. Bəzi sözlər gözlədiyiniz qədər müsbət deyil (heç olmasa bu modellə), məsələn, "good" sözü, ehtimal ki, bir çox mənfə rəylər "not good" ifadəsini ehtiva etdiyi üçün.

Əvvəlcədən Öyrədilmiş Embeddings və Dil Modellərinin Təkrar İstifadəsi

Modelin yalnız 25,000 film rəyinə əsaslanaraq faydalı söz embeddings-ləri öyrənmə bilməsi təsir edicidir. Təsəvvür edin, əgər təlim üçün milyardlarla rəyimiz olsaydı, embeddings-lər nə qədər yaxşı olardı! Təəssüf ki, bizdə yoxdur, lakin bəlkə başqa (çox) böyük mətn korpusunda (məsələn, TensorFlow Datasets-də mövcud olan Amazon rəyləri) öyrədilmiş söz embeddings-lərini təkrar istifadə edə bilərik, hətta film rəylərindən ibarət olmasa da? Axı, "amazing" sözü, istər filmlər, istərsə də hər hansı digər şey haqqında danışmaq üçün istifadə etsəniz, ümumiyyətlə eyni məna daşıyır. Üstəlik, embeddings-lər başqa bir tapşırıqda öyrədilsə belə, sentiment analysis üçün faydalı ola bilər: "awesome" və "amazing" kimi sözlər oxşar məna daşdığı üçün, hətta bir cümlədəki növbəti sözü proqnozlaşdırmaq kimi tapşırıqlarda da embedding fəzasında yığılmağa meyli olacaqlar. Əgər bütün müsbət sözlər və bütün mənfə sözlər yığınlar əmələ gətirirsə, bu, sentiment analysis üçün faydalı olacaq. Beləliklə, söz embeddings-lərini öyrətmək əvəzinə, biz sadəcə Google-un Word2vec embeddings, Stanford-un GloVe embeddings və ya Facebook-un FastText embeddings kimi əvvəlcədən öyrədilmiş embeddings-ləri yükləyib istifadə edə bilərik.

Əvvəlcədən öyrədilmiş söz embeddings-lərindən istifadə bir neçə il populyar idi, lakin bu yanaşmanın məhdudiyyətləri var. Xüsusilə, bir söz kontekstdən asılı olmayaraq tək bir təsvirə malikdir. Məsələn, "right" sözü "left and right" və "right and wrong" ifadələrində eyni şəkildə kodlaşdırılır, baxmayaraq ki, iki çox fərqli şey deməkdir. Bu məhdudiyyəti həll etmək üçün 2018-ci ildə Matthew Peters tərəfindən *Embeddings from Language Models* (ELMo) təqdim edildi: bunlar dərin ikitərəfli dil modelinin daxili vəziyyətlərindən öyrənilən kontekstləşdirilmiş söz embeddings-ləridir. Modelinizdə sadəcə əvvəlcədən öyrədilmiş embeddings-lərdən istifadə etmək əvəzinə, əvvəlcədən öyrədilmiş dil modelinin bir hissəsini təkrar istifadə edirsiniz.

Təxminən eyni vaxtda, Jeremy Howard və Sebastian Ruder tərəfindən Universal Language Model Fine-Tuning (ULMFiT) məqaləsi NLP tapşırıqları üçün unsupervised pretraining-in effektivliyini nümayiş etdirdi: müəlliflər böyük bir mətn korpusunda self-supervised learning (yəni, məlumatdan avtomatik olaraq etikətlər yaratmaq) istifadə edərək LSTM dil modeli öyrətdilər, sonra onu müxtəlif tapşırıqlarda incə tənzimlədilər. Onların modeli altı mətn təsnifatı tapşırığında ən müasir performans böyük fərqlə üstələdi (əksər hallarda səhv nisbətini 18–24% azaltdı). Üstəlik, müəlliflər göstərdilər ki, yalnız 100 etikətli nümunə üzərində incə tənzimlənmiş

əvvəlcədən öyrədilmiş model, 10,000 nümunə üzərində sıfırdan öyrədilmiş model ilə eyni performansla nail ola bilər. ULMFiT məqaləsindən əvvəl, əvvəlcədən öyrədilmiş modellərdən istifadə yalnız kompüter görməsində norma idi; NLP kontekstində pretraining yalnız söz embeddings-ləri ilə məhdudlaşdı. Bu məqalə NLP-də yeni bir dövrün başlanğıcını qeyd etdi: bu gün əvvəlcədən öyrədilmiş dil modellərini təkrar istifadə etmək normaldır.

Məsələn, 2018-ci ildə Google tədqiqatçıları komandası tərəfindən təqdim edilən Universal Sentence Encoder arxitekturasına əsaslanan bir təsnifatçı quraq. Bu model transformer arxitekturasına əsaslanır, biz bu fəslin sonrakı hissəsində baxacağıq. Rahatlıqla, model TensorFlow Hub-da mövcuddur:

```
import os
import tensorflow_hub as hub
os.environ["TFHUB_CACHE_DIR"] = "my_tfhub_cache"

model = tf.keras.Sequential([
    hub.KerasLayer("https://tfhub.dev/google/universal-sentence-encoder/4",
                   trainable=True, dtype=tf.string, input_shape=[]),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(loss="binary_crossentropy", optimizer="nadam",
              metrics=["accuracy"])

model.fit(train_set, validation_data=valid_set, epochs=10)
```

İPUCU

Bu model olduqca böyükdür – ölçüsü 1 GB-a yaxındır – buna görə yükləmək bir müddət çəkə bilər. Defolt olaraq, TensorFlow Hub modulları müvəqqəti kataloqda saxlanılır və proqramınızı hər dəfə işlətdiyiniz zaman yenidən yüklənir. Bunun qarşısını almaq üçün `TFHUB_CACHE_DIR` mühit dəyişənini seçdiyiniz bir kataloqa təyin etməlisiniz: modullar daha sonra orada saxlanılacaq və yalnız bir dəfə yüklənəcək.

Qeyd edək ki, TensorFlow Hub modul URL-inin son hissəsi modelin 4-cü versiyasını istədiyimizi göstərir. Bu versiyalaşdırma, TF Hub-da yeni bir modul versiyası buraxılırsa, modelimizi pozmayacağını təmin edir. Rahatlıqla, əgər bu URL-ni veb brauzerə daxil etsəniz, bu modul üçün sənədləri əldə edəcəksiniz.

Həmçinin qeyd edək ki, `hub.KerasLayer` yaradarkən `trainable=True` təyin etdik. Bu yolla, əvvəlcədən öyrədilmiş Universal Sentence Encoder təlim zamanı incə tənzimlənir: onun bəzi çəkirləri backprop vasitəsilə tənzimlənir. Bütün TensorFlow

Hub modulları incə tənzimləmə bilən deyil, buna görə maraqlandığınız hər bir əvvəlcədən öyrədilmiş modulun sənədlərini yoxladığınızdan əmin olun.

Təlimdən sonra, bu model 90%-dən çox validasiya dəqiqliyinə çatmalıdır. Bu, həqiqətən çox yaxşıdır: əgər tapşırığı özünüz yerinə yetirməyə çalışsanız, yəqin ki, yalnız bir az daha yaxşı edə bilərsiniz, çünki bir çox rəy həm müsbət, həm də mənfi şərtləri ehtiva edir. Bu qeyri-müəyyən rəyləri təsnif etmək qəpik atmaq kimidir.

İndiyə qədər biz char-RNN istifadə edərək mətn yaratmağa, söz səviyyəli RNN modelləri (trainable embeddings əsasında) və TensorFlow Hub-dan güclü əvvəlcədən öyrədilmiş dil modeli ilə sentiment analysis-ə baxdıq. Növbəti bölmədə biz başqa bir mühüm NLP tapşırığını – *neural machine translation* (NMT) – araşdıracağıq.

Neural Machine Translation üçün Encoder–Decoder Şəbəkəsi

Gəlin ingilis cümlələrini ispan dilinə tərcümə edəcək sadə bir NMT modeli ilə başlayaq (bax Şəkil 16-3).

Qısacası, arxitektura belədir: ingilis cümlələri encoder-a giriş kimi verilir və decoder ispan tərcümələrini çıxarır. Qeyd edək ki, ispan tərcümələri də təlim zamanı decoder-a giriş kimi istifadə olunur, lakin bir addım geri sürüşdürülür. Başqa sözlə, təlim zamanı decoder-a əvvəlki addımda *çıxarmalı olduğu* söz giriş kimi verilir, həqiqətən nə çıxardığından asılı olmayaraq. Buna *teacher forcing* deyilir – təlimi əhəmiyyətli dərəcədə sürətləndirən və modelin performansını yaxşılaşdıran bir texnika. İlk söz üçün decoder-a start-of-sequence (SOS) token-i verilir və decoder cümləni end-of-sequence (EOS) token-i ilə bitirməsi gözlənilir.

Hər bir söz əvvəlcə onun ID-si ilə təmsil olunur (məsələn, "soccer" sözü üçün 854). Sonra, Embedding qatı söz embedding-ini qaytarır. Bu söz embeddings-ləri daha sonra encoder və decoder-a verilir.

Hər bir addımda, decoder çıxış lüğətindəki (yəni ispan dilində) hər bir söz üçün bir bal çıxarır, sonra softmax aktivasiya funksiyası bu balları ehtimallara çevirir. Məsələn, ilk addımda "Me" sözü 7% ehtimala, "Yo" sözü 1% ehtimala malik ola bilər və s. Ən yüksək ehtimallı söz çıxış olur. Bu, adi bir təsnifat tapşırığına çox bənzəyir və həqiqətən də, char-RNN modelində etdiyimiz kimi, "sparse_categorical_crossentropy" loss istifadə edərək modeli öyrədə bilərsiniz.

Şəkil 16-3. Sadə bir maşın tərcüməsi modeli

Qeyd edək ki, çıxarma vaxtı (təlimdən sonra), decoder-a qidalandırmaq üçün hədəf cümləyə sahib olmayacaqsınız. Bunun əvəzinə, əvvəlki addımda çıxardığı sözü ona qidalandırmalısınız, Şəkil 16-4-də göstərildiyi kimi (bu, diaqramda göstərilməyən bir embedding axtarışını tələb edəcək).

İPUCU

2015-ci ildə Samy Bengio və digərləri tərəfindən təklif edilən məqalədə, təlim zamanı decoder-a əvvəlki *hədəf* token-i qidalandırmaqdan onu əvvəlki *çıxış* token-i ilə qidalandırmaya tədricən keçid təklif edilmişdir.

Şəkil 16-4. Çıxarma vaxtı, decoder-a əvvəlki zaman addımında çıxardığı söz giriş kimi verilir

Gəlin bu modeli quraq və öyrədək! Əvvəlcə, ingilis/ıspan cümlə cütlərindən ibarət bir dataset yükləməliyik:

```
url = "https://storage.googleapis.com/download.tensorflow.org/data/spa-eng.zip"
path = tf.keras.utils.get_file("spa-eng.zip", origin=url,
cache_dir="datasets", extract=True)

text = (Path(path).with_name("spa-eng") / "spa.txt").read_text()
```

Hər bir sətir tab ilə ayrılmış ingilis cümləsi və müvafiq ıspan tərcüməsini ehtiva edir. Biz əvvəlcə TextVectorization qatının idarə etmədiyi ıspan simvolları "ı" və "ç"-ni çıxaracağıq, sonra cümlə cütlərini analiz edəcəyik və qarışdıracağıq. Nəhayət, onları iki ayrı siyahıya, hər dil üçün birinə böləcəyik:

```
import numpy as np
text = text.replace("ı", "").replace("ç", "")
pairs = [line.split("\t") for line in text.splitlines()]
np.random.shuffle(pairs)

sentences_en, sentences_es = zip(*pairs) # separates the pairs into 2
lists
```

İlk üç cümlə cütünə baxaq:

```
>>> for i in range(3):
...     print(sentences_en[i], "=>", sentences_es[i])
...
How boring! => Qué aburrimiento!
```

I love sports. => Adoro el deporte.

Would you like to swap jobs? => Te gustaría que intercambiamos los trabajos?

Sonra, hər dil üçün iki TextVectorization qatı yaradaq və onları mətnə uyğunlaşdıraraq:

```
vocab_size = 1000
max_length = 50
text_vec_layer_en = tf.keras.layers.TextVectorization(
    vocab_size, output_sequence_length=max_length)
text_vec_layer_es = tf.keras.layers.TextVectorization(
    vocab_size, output_sequence_length=max_length)
text_vec_layer_en.adapt(sentences_en)

text_vec_layer_es.adapt([f"startofseq {s} endofseq" for s in sentences_es])
```

Burada bir neçə qeyd etmək lazımdır:

- Biz lüğət ölçüsünü 1,000 ilə məhdudlaşdırırıq, bu olduqca kiçikdir. Çünki təlim seti çox böyük deyil və kiçik bir dəyər istifadə etmək təlimi sürətləndirəcək. Ən müasir tərcümə modelləri adətən daha böyük lüğət (məsələn, 30,000), daha böyük təlim seti (gigabaytlar) və daha böyük model (yüzlərlə və hətta minlərlə meqabayt) istifadə edir. Məsələn, Helsinki Universitetinin Opus-MT modellərinə və ya Facebook-un M2M-100 modelinə baxın.
- Dataset-dəki bütün cümlələrin maksimum 50 sözü olduğu üçün, `output_sequence_length`-i 50 təyin edirik: bu yolla giriş ardıcılıqları hamısı 50 token uzunluğunda olana qədər avtomatik olaraq sıfırlarla doldurulacaq. Təlim setində 50 token-dan uzun hər hansı bir cümlə olsaydı, 50 token-a qədər qısaldılacaq.
- İspan mətni üçün, TextVectorization qatını uyğunlaşdırarkən hər bir cümləyə "startofseq" və "endofseq" əlavə edirik: bu sözləri SOS və EOS token-ləri kimi istifadə edəcəyik. Onlar həqiqi ispan sözləri olmadığı müddətcə, istənilən digər sözlərdən istifadə edə bilərsiniz.

Hər iki lüğətdəki ilk 10 token-a nəzər salaq. Onlar padding token, bilinməyən token, SOS və EOS token-ləri (yalnız ispan lüğətində), sonra isə faktiki sözlər, azalan tezliklə sıralanmış şəkildə başlayır:

```
>>> text_vec_layer_en.get_vocabulary()[:10]
['', '[UNK]', 'the', 'i', 'to', 'you', 'tom', 'a', 'is', 'he']
>>> text_vec_layer_es.get_vocabulary()[:10]
```

```
['', '[UNK]', 'startofseq', 'endofseq', 'de', 'que', 'a', 'no', 'tom', 'la']
```

Sonra, təlim seti və validasiya seti yaradaq (lazım olsa, test seti də yarada bilərsiniz). İlk 100,000 cümlə cütünü təlim üçün, qalanını validasiya üçün istifadə edəcəyik. Decoder-in girişləri ispan cümlələri üstəgəl SOS token prefiksidir. Hədəflər ispan cümlələri üstəgəl EOS şəkilçisidir:

```
X_train = tf.constant(sentences_en[:100_000])
X_valid = tf.constant(sentences_en[100_000:])
X_train_dec = tf.constant([f"startofseq {s}" for s in sentences_es[:100_000]])
X_valid_dec = tf.constant([f"startofseq {s}" for s in sentences_es[100_000:]])
Y_train = text_vec_layer_es([f"{s} endofseq" for s in sentences_es[:100_000]])
Y_valid = text_vec_layer_es([f"{s} endofseq" for s in sentences_es[100_000:]])
```

Yaxşı, indi tərcümə modelimizi qurmağa hazırıq. Model ardıcıl olmadığı üçün functional API-dən istifadə edəcəyik. O, iki mətn girişi tələb edir – biri encoder üçün, digəri decoder üçün – buna görə bununla başlayaq:

```
encoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
decoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
```

Sonra, əvvəllər hazırladığımız TextVectorization qatlarından istifadə edərək bu cümlələri kodlaşdırmalıyıq, ardınca hər dil üçün Embedding qatı ilə, `mask_zero=True` təyin edərək masking-in avtomatik idarə olunmasını təmin etməliyik. Embedding ölçüsü, həmişə olduğu kimi, tənzimləyə biləcəyiniz bir hiperparametrdir:

```
embed_size = 128

encoder_input_ids = text_vec_layer_en(encoder_inputs)
decoder_input_ids = text_vec_layer_es(decoder_inputs)

encoder_embedding_layer = tf.keras.layers.Embedding(vocab_size, embed_size,
mask_zero=True)
decoder_embedding_layer = tf.keras.layers.Embedding(vocab_size, embed_size,
mask_zero=True)

encoder_embeddings = encoder_embedding_layer(encoder_input_ids)
```

```
decoder_embeddings = decoder_embedding_layer(decoder_input_ids)
```

İPUCU

Dillər bir çox sözləri paylaşıqda, həm encoder, həm də decoder üçün eyni embedding qatından istifadə etməklə daha yaxşı performans əldə edə bilərsiniz.

İndi encoder-i yaradaq və ona embed edilmiş girişləri ötürək:

```
encoder = tf.keras.layers.LSTM(512, return_state=True)
```

```
encoder_outputs, *encoder_state = encoder(encoder_embeddings)
```

İşləri sadə saxlamaq üçün biz sadəcə bir LSTM qatından istifadə etdik, lakin onlardan bir neçəsini yığa bilərsiniz. Biz həmçinin `return_state=True` təyin edərək qatın son vəziyyətinə istinad əldə etdik. LSTM qatı istifadə etdiyimiz üçün, əslində iki vəziyyət var: qısa müddətli vəziyyət və uzun müddətli vəziyyət. Qat bu vəziyyətləri ayrı-ayrılıqda qaytarır, buna görə hər iki vəziyyəti bir siyahıda qruplaşdırmaq üçün `*encoder_state` yazmalı olduq. İndi bu (cüt) vəziyyəti decoder-in ilkin vəziyyəti kimi istifadə edə bilərik:

```
decoder = tf.keras.layers.LSTM(512, return_sequences=True)
```

```
decoder_outputs = decoder(decoder_embeddings, initial_state=encoder_state)
```

Sonra, decoder-in çıxışlarını softmax aktivasiya funksiyası ilə Dense qatından keçirərək hər bir addım üçün söz ehtimallarını əldə edə bilərik:

```
output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")
```

```
Y_proba = output_layer(decoder_outputs)
```

ÇIXIŞ QATININ OPTİMALLAŞDIRILMASI

Çıxış lüğəti böyük olduqda, hər bir mümkün söz üçün ehtimal çıxarmaq olduqca yavaş ola bilər. Əgər hədəf lüğəti 1,000 əvəzinə 50,000 ispan sözünü ehtiva etsəydi, decoder 50,000-ölçülü vektorlar çıxaracaq və belə böyük bir vektor üzərində softmax funksiyasını hesablamaq çox hesablama tələb edəcəkdi. Bunun qarşısını almaq üçün bir həll, model tərəfindən çıxarılan logits-lərə yalnız düzgün söz və təsadüfi seçilmiş səhv sözlər üçün baxmaq, sonra yalnız bu logits-lərə əsaslanan loss-un təqribini hesablamaqdır. Bu *sampled softmax* texnikası 2015-ci ildə Sébastien Jean və

digərləri tərəfindən təqdim edilmişdir. TensorFlow-da təlim zamanı `tf.nn.sampled_softmax_loss()` funksiyasından istifadə edə və çıxarma vaxtında normal softmax funksiyasından istifadə edə bilərsiniz (sampled softmax çıxarma vaxtında istifadə edilə bilməz, çünki o, hədəfi bilməyi tələb edir).

Təlimi sürətləndirmək üçün edə biləcəyiniz başqa bir şey – sampled softmax ilə uyğun gələn – çıxış qatının çəkirlərini decoder-in embedding matrisinin transpozisiyasına bağlamaqdır (Fəsil 17-də çəkirləri necə bağlayacağınızı görəcəksiniz). Bu, model parametrlərinin sayını əhəmiyyətli dərəcədə azaldır, təlimi sürətləndirir və bəzən modelin dəqiqliyini də yaxşılaşdırı bilər, xüsusən də çox təlim məlumatınız olmadıqda. Embedding matrisi, bir-hot kodlaşdırmadan sonra, bir-hot vektorlarını embedding fəzasına xəritələşdirən heç bir bias termini və aktivasiya funksiyası olmayan xətti qata bərabərdir. Çıxış qatı isə əksini edir. Beləliklə, əgər model transpozisiyası tərsinə yaxın olan bir embedding matrisi tapa bilərsə (belə bir matris *ortogonal matris* adlanır), onda çıxış qatı üçün ayrıca çəkilər dəsti öyrənməyə ehtiyac yoxdur.

Və hamısı budur! Biz sadəcə Keras Modelini yaratmalı, kompilyasiya etməli və öyrətməliyik:

```
model = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
                        outputs=[Y_proba])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
model.fit((X_train, X_train_dec), Y_train, epochs=10,
          validation_data=((X_valid, X_valid_dec), Y_valid))
```

Təlimdən sonra, modeli yeni ingilis cümlələrini ispan dilinə tərcümə etmək üçün istifadə edə bilərik. Lakin bu, `model.predict()` çağırmaq qədər sadə deyil, çünki decoder əvvəlki zaman addımında proqnozlaşdırılan sözü giriş kimi gözləyir. Bunun bir yolu, əvvəlki çıxışı izləyən və onu növbəti zaman addımında encoder-a qidalandıran xüsusi bir yaddaş hüceyrəsi yazmaqdır. Bununla belə, işləri sadə saxlamaq üçün, modeli bir neçə dəfə çağır, hər dəfə bir əlavə söz proqnozlaşdırı bilərik. Bunun üçün kiçik bir yardımçı funksiya yazaq:

```
def translate(sentence_en):
    translation = ""
    for word_idx in range(max_length):
        X = np.array([sentence_en]) # encoder input
        X_dec = np.array(["startofseq " + translation]) # decoder input
        y_proba = model.predict((X, X_dec))[0, word_idx] # last token's
        # probas
        predicted_word_id = np.argmax(y_proba)
```



```

predicted_word =
text_vec_layer_es.get_vocabulary()[predicted_word_id]
    if predicted_word == "endofseq":
        break
    translation += " " + predicted_word

return translation.strip()

```

Funksiya sadəcə bir-bir söz proqnozlaşdırmağa davam edir, tədricən tərcüməni tamamlayır və EOS token-ə çatdıqda dayanır. Gəlin sınayaq!

```
>>> translate("I like soccer")
```

```
'me gusta el fútbol'
```

Hurray, işləyir! Yaxşı, heç olmasa çox qısa cümlələrlə işləyir. Bu modelle bir müddət oynasanız, onun hələ ikidilli olmadığını və xüsusən daha uzun cümlələrlə mübarizə apardığını görəcəksiniz. Məsələn:

```
>>> translate("I like soccer and also going to the beach")
```

```
'me gusta el fútbol y a veces mismo al bus'
```

Tərcümə "I like soccer and sometimes even the bus" deyir. Beləliklə, onu necə yaxşılaşdırmaq olar? Bir yol təlim setinin ölçüsünü artırmaq və həm encoder, həm də decoder-da daha çox LSTM qatı əlavə etməkdir. Lakin bu, sizi yalnız müəyyən bir nöqtəyə qədər aparacaq, buna görə daha mürəkkəb texnikalara, ilk növbədə bidirectional recurrent qatlara baxaq.

Bidirectional RNN-lər

Hər bir zaman addımında, adi bir recurrent qat yalnız keçmiş və indiki girişlərə baxaraq çıxışını yaradır. Başqa sözlə, o, *causal* (səbəb) dir, yəni gələcəyə baxa bilməz. Bu tip RNN zaman seriyalarını proqnozlaşdırarkən və ya seq2seq modelinin decoder-ində məna kəsb edir. Lakin mətn təsnifatı və ya seq2seq modelinin encoder-i kimi tapşırıqlar üçün, çox vaxt müəyyən bir sözü kodlaşdırmazdan əvvəl növbəti sözlərə baxmaq üstünlük təşkil edir.

Məsələn, "the right arm", "the right person" və "the right to criticize" ifadələrini nəzərdən keçirin: "right" sözünü düzgün kodlaşdırmaq üçün irəli baxmaq lazımdır. Bir həll, eyni girişlər üzərində iki recurrent qat işə salmaqdır, biri sözləri soldan sağa, digəri sağdan sola oxuyur, sonra onların çıxışlarını hər zaman addımında birləşdirir, adətən concatenation etməklə. Bidirectional recurrent qatı bunu edir (bax Şəkil 16-5).

Şəkil 16-5. İkitərəfli recurrent qat

Keras-da bidirectional recurrent qatı tətbiq etmək üçün, sadəcə bir recurrent qatı `tf.keras.layers.Bidirectional` qatına sarın. Məsələn, aşağıdakı Bidirectional qatı tərcümə modelimizdə encoder kimi istifadə edilə bilər:

```
encoder = tf.keras.layers.Bidirectional(  
    tf.keras.layers.LSTM(256, return_state=True))
```

QEYD

Bidirectional qatı GRU qatının bir klonunu yaradacaq (lakin tərs istiqamətdə) və hər ikisini işlədəcək və onların çıxışlarını birləşdirəcək. Beləliklə, GRU qatı 10 vahid olsa da, Bidirectional qatı hər zaman addımında 20 dəyər çıxaracaq.

Yalnız bir problem var. Bu qat indi iki əvəzinə dörd vəziyyət qaytaracaq: irəli LSTM qatının son qısa müddətli və uzun müddətli vəziyyətləri və geri LSTM qatının son qısa müddətli və uzun müddətli vəziyyətləri. Biz bu dörd vəziyyəti birbaşa decoder-in LSTM qatının ilkin vəziyyəti kimi istifadə edə bilmərik, çünki o, yalnız iki vəziyyət (qısa müddətli və uzun müddətli) gözləyir. Biz decoder-i ikitərəfli edə bilmərik, çünki o, səbəb (causal) olmalıdır: əks halda təlim zamanı aldatmış olardı və işləməzdi. Bunun əvəzinə, biz iki qısa müddətli vəziyyəti birləşdirə və həmçinin iki uzun müddətli vəziyyəti birləşdirə bilərik:

```
encoder_outputs, *encoder_state = encoder(encoder_embeddings)  
encoder_state = [tf.concat(encoder_state[:, :2], axis=-1), # short-term (0 &  
2)  
    tf.concat(encoder_state[:, 1:2], axis=-1)] # long-term (1 &  
3)
```

İndi çıxarma vaxtında tərcümə modelinin performansını əhəmiyyətli dərəcədə yaxşılaşdırmaqla yanaşı başqa bir məşhur texnikaya baxaq: beam search.

Beam Search

Tutaq ki, bir encoder-decoder modeli öyrətmisiniz və onu "I like soccer" cümləsini ispan dilinə tərcümə etmək üçün istifadə edirsiniz. Ümid edirsiniz ki, düzgün tərcümə "me gusta el fútbol" çıxaracaq, lakin təəssüf ki, "me gustan los jugadores" çıxarır, bu da "I like the players" deməkdir. Təlim setinə baxanda, "I like cars" kimi bir çox

cümlənin "me gustan los autos" kimi tərcümə olunduğunu görürsünüz, buna görə "I like" gördükdən sonra modelin "me gustan los" çıxarması tamamilə əsassız deyildi. Təəssüf ki, bu halda səhv idi, çünki "soccer" təkdir. Model geri qayıdıb onu düzəldə bilmədi, buna görə cümləni mümkün qədər yaxşı tamamlamağa çalışdı, bu halda "jugadores" sözündən istifadə etdi. Modelə əvvəllər etdiyi səhvləri düzəltmək üçün bir şans necə verə bilərik? Ən çox yayılmış həllərdən biri *beam search* -dur: o, ən perspektivli k cümlənin qısa siyahısını saxlayır (məsələn, ən yaxşı üç) və hər bir decoder addımında onları bir sözlə genişləndirməyə çalışır, yalnız ən çox ehtimal olunan k cümləni saxlayır. k parametri *beam width* adlanır.

Məsələn, fərz edək ki, modeldən "I like soccer" cümləsini 3 beam width ilə beam search istifadə edərək tərcümə etmək üçün istifadə edirsiniz (bax Şəkil 16-6). İlk decoder addımında, model tərcümə cümləsindəki hər bir mümkün ilk söz üçün təxmin edilən ehtimalı çıxaracaq. Tutaq ki, ən yaxşı üç söz "me" (75% təxmin edilən ehtimal), "a" (3%) və "como" (1%)-dur. Bu, indiyə qədər bizim qısa siyahımızdır. Sonra, hər bir cümlə üçün növbəti sözün tapmaq üçün modeli istifadə edirik. Birinci cümlə ("me") üçün, bəlkə model "gustan" sözü üçün 36%, "gusta" sözü üçün 32%, "encanta" sözü üçün 16% ehtimal çıxarır və s. Qeyd edək ki, bunlar əslində cümlənin "me" ilə başladığını nəzərə alaraq *şərti* ehtimallardır. İkinci cümlə ("a") üçün, model "mi" sözü üçün 50% *şərti* ehtimal çıxara bilər və s. Lügətin 1,000 sözü olduğunu fərz etsək, hər cümlə üçün 1,000 ehtimal əldə edəcəyik.

Sonra, nəzərdən keçirdiyimiz 3,000 iki sözlü cümlənin ($3 \times 1,000$) hər birinin ehtimallarını hesablayırıq. Bunu hər bir sözün təxmin edilən *şərti* ehtimalını tamamladığı cümlənin təxmin edilən ehtimalına vurmaqla edirik. Məsələn, "me" cümləsinin təxmin edilən ehtimalı 75% idi, "gustan" sözünün (birinci sözün "me" olduğunu nəzərə alaraq) təxmin edilən *şərti* ehtimalı 36% idi, buna görə "me gustan" cümləsinin təxmin edilən ehtimalı $75\% \times 36\% = 27\%$ -dir. Bütün 3,000 iki sözlü cümlənin ehtimallarını hesabladıqdan sonra, yalnız ən yaxşı 3-ü saxlayırıq. Bu nümunədə hamısı "me" sözü ilə başlayır: "me gustan" (27%), "me gusta" (24%) və "me encanta" (12%). İndi "me gustan" cümləsi qalıbdır, lakin "me gusta" hələ aradan qaldırılmayıb.

Şəkil 16-6. Beam search, beam width 3 ilə

Sonra eyni prosesi təkrar edirik: modeldən bu üç cümlənin hər birində növbəti sözün proqnozlaşdırmaq üçün istifadə edirik və nəzərdən keçirdiyimiz bütün 3,000 üç sözlü cümlənin ehtimallarını hesablayırıq. Bəlkə ən yaxşı üç indi "me gustan los" (10%), "me gusta el" (8%) və "me gusta mucho" (2%) olur. Növbəti addımda "me gusta el fútbol" (6%), "me gusta mucho el" (1%) və "me gusta el deporte" (0.2%) əldə edə bilərik. Diqqət yetirin ki, "me gustan" aradan qaldırıldı və düzgün tərcümə indi qabaqdadır. Biz heç bir əlavə təlim olmadan, sadəcə daha ağıllı istifadə etməklə encoder-decoder modelimizin performansını artırdıq.

İPUCU

TensorFlow Addons kitabxanası, attention, beam search və daha çoxu daxil olmaqla encoder-decoder modelləri qurmaq üçün tam seq2seq API ehtiva edir. Lakin onun sənədləri hazırda çox məhduddur. Beam search tətbiq etmək yaxşı bir məşqdır, buna görə sınayın! Mümkün bir həll üçün bu fəslin notebook-na baxın.

Bütün bunlarla, kifayət qədər qısa cümlələr üçün ağlabatan yaxşı tərcümələr əldə edə bilərsiniz. Təəssüf ki, bu model uzun cümlələri tərcümə etməkdə həqiqətən pis olacaq. Bir daha, problem RNN-lərin məhdud qısa müddətli yaddaşından qaynaqlanır. *Attention mechanisms* (diqqət mexanizmləri) bu problemi həll edən oyun dəyişdirən yenilikdir.

Attention Mechanisms

Şəkil 16-3-də "soccer" sözündən onun tərcüməsi "fútbol"-a qədər olan yolu nəzərdən keçirin: olduqca uzundur! Bu o deməkdir ki, bu sözün (bütün digər sözlərlə birlikdə) təsviri həqiqətən istifadə olunmazdan əvvəl çox sayda addım daşınmalıdır. Bu yolu daha qısa edə bilmərikmi?

Bu, 2014-cü ildə Dzmitry Bahdanau və digərlərinin əlamətdar məqaləsindəki əsas fikir idi, burada müəlliflər decoder-in hər bir zaman addımında müvafiq sözlərə (encoder tərəfindən kodlaşdırılmış) diqqət yetirməsinə imkan verən bir texnika təqdim etdilər. Məsələn, decoder "fútbol" sözünü çıxarmalı olduğu zaman addımında, o, "soccer" sözünə diqqətini yönəldəcək. Bu o deməkdir ki, giriş sözündən onun tərcüməsinə qədər olan yol indi çox qısadır, buna görə RNN-lərin qısa müddətli yaddaş məhdudiyyətlərinin təsiri daha az olur. Attention mechanisms neural machine translation (və ümumiyyətlə deep learning) sahəsində inqilab etdi, xüsusən uzun cümlələr üçün (məsələn, 30 sözdən çox) ən müasir vəziyyətin əhəmiyyətli dərəcədə yaxşılaşmasına imkan verdi.

QEYD

NMT-də ən çox istifadə olunan metrika *bilingual evaluation understudy* (BLEU) balıdır, o, model tərəfindən istehsal edilən hər bir tərcüməni insanlar tərəfindən istehsal edilən bir neçə yaxşı tərcümə ilə müqayisə edir: hədəf tərcümələrin hər hansı birində görünən n -grams (n sözlü ardıcılıqlar) sayını hesablayır və istehsal olunan n -grams-ların hədəf tərcümələrdəki tezliyini nəzərə almaq üçün balı tənzimləyir.

Şəkil 16-7 əlavə edilmiş attention mexanizmi ilə encoder–decoder modelimizi göstərir. Solda, encoder və decoder var. Sadəcə encoder-in son gizli vəziyyətini decoder-a göndərmək əvəzinə, həmçinin hər addımda əvvəlki hədəf sözü (hələ də edilir, baxmayaraq ki, şəkildə göstərilmir), biz indi bütün encoder çıxışlarını da decoder-a göndəririk. Decoder bütün bu encoder çıxışlarını bir anda idarə edə bilmədiyi üçün, onların birləşdirilməsi lazımdır: hər bir zaman addımında, decoder-in yaddaş hüceyrəsi bütün encoder çıxışlarının çəkili cəmini hesablayır. Bu, onun bu addımda hansı sözlərə diqqət yetirəcəyini müəyyən edir. Çəki $\alpha(t, i)$ i ci encoder çıxışının t ci decoder zaman addımındakı çəkisidir. Məsələn, əgər çəki $\alpha(3,2)$ $\alpha(3,0)$ və $\alpha(3,1)$ çəkilərindən çox böyükdürsə, decoder 2 nömrəli söz ("soccer") üçün encoder çıxışına digər iki çıxışdan daha çox diqqət yetirəcək, heç olmasa bu zaman addımında. Decoder-in qalan hissəsi əvvəlki kimi işləyir: hər bir zaman addımında yaddaş hüceyrəsi indi müzakirə etdiyimiz girişləri, üstəlik əvvəlki zaman addımındakı gizli vəziyyəti alır və nəhayət (diqramda təsvir edilməsə də) əvvəlki zaman addımındakı hədəf sözü (və ya çıxarma vaxtında, əvvəlki zaman addımının çıxışı) alır.

Şəkil 16-7. Attention modeli ilə encoder–decoder şəbəkəsi istifadə edərək neural machine translation

Bəs bu $\alpha(t, i)$ çəkiləri haradan gəlir? Onlar *alignment model* (və ya *attention layer*) adlanan kiçik bir neyron şəbəkə tərəfindən yaradılır, bu, encoder–decoder modelinin qalan hissəsi ilə birlikdə öyrədilir. Bu alignment model Şəkil 16-7-nin sağ tərəfində təsvir edilmişdir. O, hər bir encoder çıxışını decoder-in əvvəlki gizli vəziyyəti ilə birlikdə emal edən tək neyronlu Dense qatı ilə başlayır. Bu qat hər bir encoder çıxışı üçün bir bal (və ya enerji) çıxarır: bu bal hər bir çıxışın decoder-in əvvəlki gizli vəziyyəti ilə nə qədər uyğunlaşdığını ölçür. Məsələn, Şəkil 16-7-də model artıq "me gusta el" (yəni "I like") çıxarmışdır, buna görə indi bir isim gözləyir: "soccer" sözü cari vəziyyətə ən yaxşı uyğun gələn sözdür, buna görə yüksək bal alır. Nəhayət, bütün ballar softmax qatından keçərək hər bir encoder çıxışı üçün son α çəkisi alınır. Verilmiş bir decoder zaman addımı üçün bütün çəkilər 1-ə qədər toplanır. Bu xüsusi attention mexanizmi *Bahdanau attention* adlanır (2014-cü il məqaləsinin ilk müəllifinin adı ilə). O, encoder çıxışını decoder-in əvvəlki gizli vəziyyəti ilə birləşdirdiyi üçün bəzən *concatenative attention* (və ya *additive attention*) adlanır.

QEYD

Əgər giriş cümləsi n söz uzunluğundadırsa və çıxış cümləsi təxminən eyni uzunluqdadırsa, bu model təxminən $_n^2$ çəki hesablamalı olacaq. Xoşbəxtlikdən, bu kvadratik hesablama mürəkkəbliyi hələ də əlçatandır, çünki hətta uzun cümlələrin minlərlə sözü yoxdur.

Digər bir ümumi attention mexanizmi, *Luong attention* və ya *multiplicative attention* kimi tanınır, 2015-ci ildə Minh-Thang Luong və digərləri tərəfindən təklif edilmişdir. Alignment modelinin məqsədi encoder çıxışlarından biri ilə decoder-in əvvəlki gizli vəziyyəti arasındakı oxşarlığı ölçmək olduğundan, müəlliflər sadəcə bu iki vektorun nöqtə hasilini (bax Fəsil 4) hesablamağı təklif etdilər, çünki bu, çox vaxt kifayət qədər yaxşı bir oxşarlıq ölçüsüdür və müasir aparatlar onu çox səmərəli hesablaya bilər. Bunun mümkün olması üçün hər iki vektor eyni ölçülülüyə malik olmalıdır. Nöqtə hasilini bir bal verir və bütün ballar (müəyyən bir decoder zaman addımında) Bahdanau attention-da olduğu kimi softmax qatından keçərək son çəkilişi verir. Luong və digərlərinin təklif etdiyi başqa bir sadələşdirmə, decoder-in gizli vəziyyətini əvvəlki zaman addımı əvəzinə cari zaman addımında istifadə etmək (yəni $h(t)$ əvəzinə $h(t-1)$), sonra attention mexanizminin çıxışını ($\tilde{h}(t)$ qeyd olunur) birbaşa decoder-in proqnozlarını hesablamaq üçün istifadə etməkdir, onu decoder-in cari gizli vəziyyətini hesablamaq üçün istifadə etmək əvəzinə. Tədqiqatçılar həmçinin nöqtə hasil mexanizminin bir variantını təklif etdilər, burada encoder çıxışları nöqtə hasilləri hesablanmazdan əvvəl tam əlaqəli qatdan (bias termini olmadan) keçir. Buna "general" nöqtə hasil yanaşması deyilir. Tədqiqatçılar hər iki nöqtə hasil yanaşmasını concatenative attention mexanizmi ilə (v yenidən miqyaslandırma parametr vektoru əlavə etməklə) müqayisə etdilər və nöqtə hasil variantlarının concatenative attention-dan daha yaxşı performans göstərdiyini müşahidə etdilər. Bu səbəbdən, concatenative attention indi daha az istifadə olunur. Bu üç attention mexanizminin tənlikləri Tənlik 16-1-də xülasə edilmişdir.

Tənlik 16-1. Attention mexanizmləri

$$\tilde{h}(t) = \sum_i \alpha(t,i) y(i)$$

$$\alpha(t,i) = \exp(e(t,i)) / \sum_i' \exp(e(t,i'))$$

$$e(t,i) =$$

$$h(t)^T y(i) \quad (\text{dot})$$

$$h(t)^T W y(i) \quad (\text{general})$$

$$v^T \tanh(W [h(t); y(i)]) \quad (\text{concat})$$

Keras Luong attention üçün `tf.keras.layers.Attention` qatını və Bahdanau attention üçün `AdditiveAttention` qatını təqdim edir. Gəlin encoder-decoder modelimizə Luong attention əlavə edək. Attention qatına bütün encoder çıxışlarını ötürməmiz lazım olacağı üçün, əvvəlcə encoder-i yaradarkən `return_sequences=True` təyin etməliyik:

```
encoder = tf.keras.layers.Bidirectional(
```

```
tf.keras.layers.LSTM(256, return_sequences=True, return_state=True))
```

Sonra, attention qatını yaratmalı və ona decoder-in vəziyyətlərini və encoder çıxışlarını ötürməliyik. Lakin, hər bir addımda decoder-in vəziyyətlərinə daxil olmaq üçün xüsusi yaddaş hüceyrəsi yazmalı olardıq. Sadəlik üçün, decoder-in çıxışlarını onun vəziyyətləri əvəzinə istifadə edək: praktikada bu da yaxşı işləyir və kodlaşdırmaq daha asandır. Sonra attention qatının çıxışlarını birbaşa çıxış qatına ötürürük, Luong attention məqaləsində təklif edildiyi kimi:

```
attention_layer = tf.keras.layers.Attention()
attention_outputs = attention_layer([decoder_outputs, encoder_outputs])
output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")

Y_proba = output_layer(attention_outputs)
```

Və hamısı budur! Bu modeli öyrətsəniz, onun indi daha uzun cümlələri idarə etdiyini görəcəksiniz. Məsələn:

```
>>> translate("I like soccer and also going to the beach")

'me gusta el fútbol y también ir a la playa'
```

Qısacası, attention qatı modelin diqqətini girişin bir hissəsinə yönəltmək üçün bir yol təqdim edir. Lakin bu qatı başqa bir şəkildə də düşünmək olar: o, diferensiallana bilən yaddaş axtarış mexanizmi kimi çıxış edir.

Məsələn, fərz edək ki, encoder "I like soccer" giriş cümləsini təhlil etdi və "I" sözünün subyekt, "like" sözünün isə fel olduğunu anlamağı bacardı, buna görə bu məlumatı bu sözlər üçün çıxışlarında kodlaşdırdı. İndi fərz edək ki, decoder subyekti artıq tərcümə etdi və indi növbəti feli tərcümə etməli olduğunu düşünür. Bunun üçün, o, giriş cümləsindən feli almaq lazımdır. Bu, bir lüğət axtarışına bənzəyir: sanki encoder {"subject": "They", "verb": "played", ...} lüğətini yaratmışdı və decoder "verb" açarına uyğun dəyəri axtarmaq istəyirdi.

Lakin modelin açarları təmsil etmək üçün diskret token-ləri yoxdur (məsələn, "subject" və ya "verb"); əvəzində, o, təlim zamanı öyrəndiyi bu konseptlərin vektorlaşdırılmış təsvirlərinə malikdir, buna görə axtarış üçün istifadə edəcəyi sorğu lüğətdəki heç bir açar ilə mükəmməl uyğunlaşmayacaq. Həll, sorğu ilə lüğətdəki hər bir açar arasında oxşarlıq ölçüsünü hesablamaq, sonra bu oxşarlıq ballarını 1-ə qədər toplanan çəkirlərə çevirmək üçün softmax funksiyasından istifadə etməkdir. Daha əvvəl gördüyümüz kimi, attention qatı məhz bunu edir. Əgər feli təmsil edən açar sorğuya ən çox bənzəyirsə, həmin açarın çəkisi 1-ə yaxın olacaq.

Sonra, attention qatı müvafiq dəyərlərin çəkili cəmini hesablayır: əgər "verb" açarının çəkisi 1-ə yaxındırsa, çəkili cəm "played" sözünün təsvirinə çox yaxın olacaq.

Buna görə Keras Attention və AdditiveAttention qatları hər ikisi giriş olaraq iki və ya üç elementi ehtiva edən bir siyahı gözləyir: *queries* (sorgular), *keys* (açarlar) və istəgə bağlı olaraq *values* (dəyərlər). Əgər heç bir dəyər ötürməsəniz, onlar avtomatik olaraq açarlara bərabər olur. Beləliklə, əvvəlki kod nümunəsinə yenidən baxsaq, decoder çıxışları sorgulardır və encoder çıxışları həm açarlar, həm də dəyərlərdir. Hər bir decoder çıxışı (yəni hər bir sorğu) üçün, attention qatı decoder çıxışına ən çox bənzəyən encoder çıxışlarının (yəni açarlar/dəyərlər) çəkili cəmini qaytarır.

Nəticə etibarilə, attention mexanizmi öyrənilə bilən yaddaş axtarış sistemidir. O, o qədər güclüdür ki, əslində yalnız attention mexanizmlərindən istifadə edərək müasir modellər qura bilərsiniz. Transformer arxitekturasına daxil olun.

Attention Is All You Need: Orijinal Transformer Arxitekturası

2017-ci ildəki əlamətdar bir məqalədə, Google tədqiqatçıları komandası "Attention Is All You Need" (Yalnız Diqqət Lazımdır) təklif etdi. Onlar *transformer* adlanan bir arxitektura yaratdılar, bu, heç bir recurrent və ya convolutional qat istifadə etmədən (sadəcə attention mexanizmləri, embedding qatları, dense qatları, normalization qatları və bir neçə digər komponent) NMT-də ən müasir vəziyyəti əhəmiyyətli dərəcədə yaxşılaşdırdı. Model recurrent olmadığı üçün, RNN-lər kimi vanishing və ya exploding gradients problemlərindən o qədər də əziyyət çəkmir, daha az addımda öyrədilə bilər, çoxsaylı GPU-lar üzərində paralelləşdirmək daha asandır və RNN-lərdən daha yaxşı uzunmüddətli naxışları tuta bilər. Orijinal 2017 transformer arxitekturası Şəkil 16-8-də təsvir edilmişdir.

Qısacası, Şəkil 16-8-in sol hissəsi encoder, sağ hissəsi isə decoder-dir. Hər bir embedding qatı [batch size, sequence length, embedding size] formasında 3D tensor çıxarır. Bundan sonra, tensorlar transformer vasitəsilə axdıqca tədricən transformasiya olunur, lakin onların forması eyni qalır.

Şəkil 16-8. Orijinal 2017 transformer arxitekturası

Əgər transformer-i NMT üçün istifadə etsəniz, təlim zamanı ingilis cümlələrini encoder-a və müvafiq ispan tərcümələrini decoder-a qidalandırmalısınız, hər cümlənin əvvəlinə əlavə SOS token-i daxil etməklə. Çıxarma vaxtında, transformer-i bir neçə dəfə çağırmalı, tərcümələri bir-bir söz istehsal etməli və hər dəfə qismən tərcümələri decoder-a qidalandırmalısınız, əvvəllər `translate()` funksiyasında etdiyimiz kimi.

Encoder-in rolu girişləri – ingilis cümləsinin söz təsvirlərini – hər bir sözün təsviri sözün mənasını cümlə kontekstində mükəmməl tutana qədər tədricən transformasiya etməkdir. Məsələn, əgər encoder-a "I like soccer" cümləsini versəniz, "like" sözü olduqca qeyri-müəyyən bir təsvirlə başlayacaq, çünki bu söz müxtəlif kontekstlərdə fərqli şeylər ifadə edə bilər: "I like soccer" ilə "It's like that" i düşünün. Lakin encoder-dən keçdikdən sonra, sözün təsviri verilmiş cümlədə "like" sözünün düzgün mənasını (yəni, xoşlamaq) və tərcümə üçün tələb oluna biləcək hər hansı digər məlumatı (məsələn, fel olduğunu) tutmalıdır.

Decoder-in rolu tərcümə cümləsindəki hər bir söz təsvirini tərcümədəki növbəti sözün təsvirinə tədricən çevirməkdir. Məsələn, tərcümə ediləcək cümlə "I like soccer"-dursa və decoder-in giriş cümləsi "<SOS> me gusta el fútbol"-dursa, decoder-dən keçdikdən sonra "el" sözünün söz təsviri "fútbol" sözünün təsvirinə çevriləcək. Eynilə, "fútbol" sözünün təsviri EOS token-inin təsvirinə çevriləcək.

Decoder-dən keçdikdən sonra, hər bir söz təsviri softmax aktivasiya funksiyası ilə son Dense qatından keçir və ümid edirik ki, düzgün növbəti söz üçün yüksək ehtimal, bütün digər sözlər üçün aşağı ehtimal çıxaracaq. Proqnozlaşdırılan cümlə "me gusta el fútbol <EOS>" olmalıdır.

Bu, ümumi mənzərə idi; indi Şəkil 16-8-i daha təfərrüatlı nəzərdən keçirək:

- Əvvəlcə, həm encoder, həm də decoder N dəfə yığılmış modulları ehtiva edir. Məqalədə $N = 6$. Bütün encoder yığınının son çıxışları decoder-a bu N səviyyənin hər birində verilir.
- Yaxınlaşdıqda, əksər komponentlərlə artıq tanış olduğunuzu görə bilərsiniz: iki embedding qatı; bir neçə skip connection, hər biri layer normalization qatı ilə; hər biri iki dense qatından ibarət bir neçə feedforward modul (birincisi ReLU aktivasiya funksiyası ilə, ikincisi aktivasiya funksiyası olmadan); və nəhayət çıxış qatı softmax aktivasiya funksiyası ilə dense qatıdır. Lazım gələrsə, attention qatlarından və feedforward modullarından sonra bir az dropout da əlavə edə bilərsiniz. Bütün bu qatlar time-distributed olduğundan, hər bir söz bütün digərlərindən müstəqil olaraq emal olunur. Bəs sözləri tamamilə ayrı-ayrı baxaraq cümləni necə tərcümə edə bilərik? Yaxşı, edə bilmərik, buna görə yeni komponentlər işə düşür:
 - Encoder-in *multi-head attention* qatı eyni cümlədəki bütün digər sözlərə diqqət yetirməklə hər bir söz təsvirini yeniləyir. "like" sözünün qeyri-müəyyən təsviri verilmiş cümlədə onun dəqiq mənasını tutan daha zəngin və daha dəqiq təsvirə çevrildiyi yerdir. Bunun dəqiq necə işlədiyini qısa müddətdə müzakirə edəcəyik.
 - Decoder-in *masked multi-head attention* qatı eyni şeyi edir, lakin bir sözü emal edərkən ondan sonra yerləşən sözlərə diqqət yetirmir: bu, səbəb (causal) qatıdır. Məsələn, "gusta" sözünü emal edərkən, yalnız "<SOS>

me gusta" sözlərinə diqqət yetirir və "el fútbol" sözlərini nəzər almır (əks halda aldatmaq olardı).

- Decoder-in yuxarı *multi-head attention* qatı decoder-in ingilis cümləsindəki sözlərə diqqət yetirdiyi yerdir. Bu, öz-özünə diqqət deyil, *cross-attention* adlanır. Məsələn, decoder "el" sözünü emal edərkən və onun təsvirini "fútbol" sözünün təsvirinə çevirərkən, yəqin ki, "soccer" sözünə çox diqqət yetirəcək.
- *Positional encodings* (mövqe kodlaşdırmaları) hər bir sözün cümlədəki mövqeyini təmsil edən sıx vektorlardır (söz embeddings-ləri kimi). n ci söz üçün mövqe kodlaşdırması hər bir cümlədəki n ci sözün söz embedding-inə əlavə edilir. Bu, transformer arxitekturasındakı bütün qatlar söz mövqələrini nəzərə almadığı üçün lazımdır: mövqe kodlaşdırmaları olmadan, giriş ardıcılıqlarını qarışdırma bilərsiniz və o, çıxış ardıcılıqlarını eyni şəkildə qarışdıracaq. Aydındır ki, sözlərin sırası vacibdir, buna görə transformer-a mövqe məlumatını hansısa şəkildə verməliyik: söz təsvirlərinə mövqe kodlaşdırmalarını əlavə etmək buna nail olmaq üçün yaxşı bir yoldur.

QEYD

Şəkil 16-8-də hər bir multi-head attention qatına gedən ilk iki ox açarları və dəyərləri, üçüncü ox isə sorğuları təmsil edir. Self-attention qatlarında hər üçü əvvəlki qat tərəfindən çıxarılan söz təsvirlərinə bərabərdir, decoder-in yuxarı attention qatında isə açarlar və dəyərlər encoder-in son söz təsvirlərinə, sorğular isə əvvəlki qat tərəfindən çıxarılan söz təsvirlərinə bərabərdir.

Transformer arxitekturasının yeni komponentlərini daha təfərrüatlı nəzərdən keçirək, mövqe kodlaşdırmalarından başlayaraq.

Mövqe kodlaşdırmaları (Positional encodings)

Mövqe kodlaşdırması bir cümlə daxilindəki sözün mövqeyini kodlaşdıran sıx vektordur: i ci mövqe kodlaşdırması cümlədəki i ci sözün söz embedding-inə əlavə edilir. Bunu tətbiq etməyin ən asan yolu Embedding qatından istifadə etmək və onu batch-dəki maksimum ardıcılıq uzunluğuna qədər 0-dan bütün mövqələri kodlaşdırmaq, sonra nəticəni söz embeddings-lərinə əlavə etməkdir. Broadcasting qaydaları mövqe kodlaşdırmalarının hər bir giriş ardıcılığına tətbiq olunmasını təmin edəcək. Məsələn, encoder və decoder girişlərinə mövqe kodlaşdırmalarını necə əlavə etmək olar:

```
max_length = 50 # max length in the whole training set
```

```

embed_size = 128
pos_embed_layer = tf.keras.layers.Embedding(max_length, embed_size)

batch_max_len_enc = tf.shape(encoder_embeddings)[1]
encoder_in = encoder_embeddings + pos_embed_layer(tf.range(batch_max_len_enc))

batch_max_len_dec = tf.shape(decoder_embeddings)[1]
decoder_in = decoder_embeddings + pos_embed_layer(tf.range(batch_max_len_dec))

```

Qeyd edək ki, bu tətbiq embeddings-lərin adi tensorlar kimi təmsil olunduğunu fərz edir, ragged tensorlar kimi yox. Encoder və decoder mövqe kodlaşdırmaları üçün eyni Embedding qatını paylaşır, çünki onlar eyni embedding ölçüsünə malikdir (çox vaxt belə olur).

Trainable mövqe kodlaşdırmaları əvəzinə, transformer məqaləsinin müəllifləri müxtəlif tezliklərdə sine və kosinus funksiyalarına əsaslanan sabit mövqe kodlaşdırmalarını seçdilər. Mövqe kodlaşdırma matrisi P Tənlik 16-2-də müəyyən edilmişdir və Şəkil 16-9-un yuxarı hissəsində (transpozisiya olunmuş) təsvir edilmişdir, burada $P_{p,i}$ cümlədəki p ci mövqedə yerləşən söz üçün kodlaşdırmanın i ci komponentidir.

Tənlik 16-2. Sine/kosinus mövqe kodlaşdırmaları

$P_{p,i} = \sin(p / 10000^{(i/d)})$ əgər i cüt
 $P_{p,i} = \cos(p / 10000^{((i-1)/d)})$ əgər i tək

Şəkil 16-9. Sine/kosinus mövqe kodlaşdırma matrisi (transpozisiya olunmuş, yuxarı) i -nin iki dəyərində diqqət yetirməklə (aşağı)

Bu həll trainable mövqe kodlaşdırmaları ilə eyni performans verə bilər və modelə heç bir parametr əlavə etmədən özbaşına uzun cümlələrə qədər genişlənə bilər (lakin çoxlu pretraining məlumatı olduqda, trainable mövqe kodlaşdırmaları adətən üstünlük təşkil edir). Bu mövqe kodlaşdırmaları söz embeddings-lərinə əlavə edildikdən sonra, modelin qalan hissəsi hər bir sözün cümlədəki mütləq mövqeyinə çıxış əldə edir, çünki hər mövqe üçün unikal bir mövqe kodlaşdırması var (məsələn, cümlədəki 22-ci mövqedə yerləşən söz üçün mövqe kodlaşdırması Şəkil 16-9-un yuxarı solunda şaquli kəsikli xətt ilə təmsil olunur və onun həmin mövqe üçün unikal olduğunu görə bilərsiniz). Üstəlik, salınan funksiyaların (sine və kosinus) seçimi modelin nisbi mövqeləri də öyrənməsinə imkan verir. Məsələn, 38 söz aralığında yerləşən sözlər (məsələn, $p = 22$ və $p = 60$) hər zaman $i = 100$ və $i = 101$ kodlaşdırma ölçülərində eyni mövqe kodlaşdırma dəyərlərinə malikdir, Şəkil 16-9-da görə

bilərsiniz. Bu, hər bir tezlik üçün həm sine, həm də kosinusa ehtiyac olduğunu izah edir: əgər yalnız sine ($i = 100$ -də mavi dalğa) istifadə etsəydik, model $p = 22$ və $p = 35$ mövqələrini ayırd edə bilməzdi (xaç ilə qeyd olunmuşdur).

TensorFlow-da PositionalEncoding qatı yoxdur, lakin onu yaratmaq çox çətin deyil. Səmərəlilik səbəbləri üçün, mövqe kodlaşdırma matrisini konstruktorda əvvəlcədən hesablayırıq. `call()` metodu bu kodlaşdırma matrisini giriş ardıcılıqlarının maksimum uzunluğuna qədər qısaldır və onları girişlərə əlavə edir. Biz həmçinin `supports_masking=True` təyin edirik ki, girişin avtomatik maskası növbəti qata ötürülsün:

```
class PositionalEncoding(tf.keras.layers.Layer):
    def __init__(self, max_length, embed_size, dtype=tf.float32, **kwargs):
        super().__init__(dtype=dtype, **kwargs)
        assert embed_size % 2 == 0, "embed_size must be even"
        p, i = np.meshgrid(np.arange(max_length), 2 * np.arange(embed_size
// 2))
        pos_emb = np.empty((1, max_length, embed_size))
        pos_emb[0, :, ::2] = np.sin(p / 10_000 ** (i / embed_size)).T
        pos_emb[0, :, 1::2] = np.cos(p / 10_000 ** (i / embed_size)).T
        self.pos_encodings = tf.constant(pos_emb.astype(self.dtype))
        self.supports_masking = True
    def call(self, inputs):
        batch_max_length = tf.shape(inputs)[1]

        return inputs + self.pos_encodings[:, :batch_max_length]
```

Gəlin bu qatdan encoder girişlərinə mövqe kodlaşdırmasını əlavə etmək üçün istifadə edək:

```
pos_embed_layer = PositionalEncoding(max_length, embed_size)
encoder_in = pos_embed_layer(encoder_embeddings)
```

```
decoder_in = pos_embed_layer(decoder_embeddings)
```

İndi transformer modelinin ürəyi olan multi-head attention qatına daha dərinə baxaq.

Multi-head attention

Multi-head attention qatının necə işlədiyini anlamaq üçün əvvəlcə onun əsaslandığı *scaled dot-product attention* qatını başa düşməliyik. Onun tənliyi Tənlik 16-3-də

vektorlaşdırılmış formada göstərilmişdir. Bu, Luong attention ilə eynidir, bir miqyaslama amili istisna olmaqla.

Tənlilik 16-3. Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_{\text{keys}}}) V$$

Bu tənlildə:

- Q hər bir *query* üçün bir sıra ehtiva edən matrisdir. Onun forması $[n_{\text{queries}}, d_{\text{keys}}]$ -dir, burada n_{queries} sorğuların sayı və d_{keys} hər bir sorğu və hər bir açarın ölçülərinin sayıdır.
- K hər bir *key* üçün bir sıra ehtiva edən matrisdir. Onun forması $[n_{\text{keys}}, d_{\text{keys}}]$ -dir, burada n_{keys} açarların və dəyərlərin sayıdır.
- V hər bir *value* üçün bir sıra ehtiva edən matrisdir. Onun forması $[n_{\text{keys}}, d_{\text{values}}]$ -dir, burada d_{values} hər bir dəyərin ölçülərinin sayıdır.
- QK^T -nin forması $[n_{\text{queries}}, n_{\text{keys}}]$ -dir: o, hər bir sorğu/açar cütü üçün bir oxşarlıq balını ehtiva edir. Bu matrisin çox böyük olmasının qarşısını almaq üçün giriş ardıcılıqları çox uzun olmamalıdır (bu məhdudiyyəti necə aşacağımızı bu fəslin sonrakı hissəsində müzakirə edəcəyik). Softmax funksiyasının çıxışı eyni formaya malikdir, lakin bütün sıralar 1-ə qədər toplanır. Son çıxış $[n_{\text{queries}}, d_{\text{values}}]$ formasına malikdir: hər bir sorğu üçün bir sıra var, burada hər bir sıra sorğu nəticəsini (dəyərlərin çəkili cəmi) təmsil edir.
- Miqyaslama amili $1 / \sqrt{d_{\text{keys}}}$ oxşarlıq ballarını aşağı miqyaslandıraraq softmax funksiyasının doymasının qarşısını alır, bu da kiçik qradientlərə səbəb olardı.

Bəzi açar/dəyər cütlərini, softmax-ı hesablamazdan dərhal əvvəl müvafiq oxşarlıq ballarına çox böyük mənfi dəyər əlavə etməklə maskalamaq mümkündür. Bu, masked multi-head attention qatında faydalıdır.

`tf.keras.layers.Attention` qatı yaradarkən `use_scale=True` təyin etsəniz, o, oxşarlıq ballarını necə düzgün aşağı miqyaslandıracağını öyrənən əlavə bir parametr yaradacaq. Transformer modelində istifadə olunan scaled dot-product attention demək olar ki, eynidir, yalnız hər zaman oxşarlıq ballarını eyni amil, $1 / \sqrt{d_{\text{keys}}}$ ilə miqyaslandırır.

Qeyd edək ki, Attention qatının girişləri Q, K və V kimidir, lakin əlavə batch ölçüsü (birinci ölçü) ilə. Daxili olaraq, qat `tf.matmul(queries, keys)`-ə bir çağırışla batch-dəki bütün cümlələr üçün bütün attention ballarını hesablayır: bu onu son dərəcə səmərəli edir. Həqiqətən, TensorFlow-da, əgər A və B ikidən çox ölçüyə malik tensorlardırsa – məsələn, müvafiq olaraq $[2, 3, 4, 5]$ və $[2, 3, 5, 6]$ formalı – onda `tf.matmul(A, B)` bu tensorları 2×3 massivlər kimi qəbul edəcək, burada hər bir

hüceyrə bir matris ehtiva edir və müvafiq matrisləri vuracaq: A-da i ci sıra və j ci sütundakı matris B-də i ci sıra və j ci sütundakı matrisə vurulacaq. 4×5 matrisin 5×6 matrisə hasili 4×6 matris olduğundan, `tf.matmul(A, B)` [2, 3, 4, 6] formasında massiv qaytaracaq.

İndi multi-head attention qatına baxmağa hazırıq. Onun arxitekturası Şəkil 16-10-da göstərilmişdir.

Şəkil 16-10. Multi-head attention qatının arxitekturası

Gördüyünüz kimi, bu, sadəcə bir dəstə scaled dot-product attention qatıdır, hər biri dəyərlərin, açarların və sorğuların xətti transformasiyasından (yəni, aktivasiya funksiyası olmayan time-distributed dense qatı) əvvəl gəlir. Bütün çıxışlar sadəcə birləşdirilir və son xətti transformasiyadan (yenə time-distributed) keçir.

Bəs niyə? Bu arxitekturanın arxasında duran intuitiv fikir nədir? Yaxşı, "I like soccer" cümləsindəki "like" sözünü bir daha nəzərdən keçirək. Encoder onun fel olduğunu kodlaşdırmaq üçün kifayət qədər ağıllı idi. Lakin söz təsviri həmçinin mövqe kodlaşdırmaları sayəsində mətndəki mövqeyini və yaqin ki, tərcüməsi üçün faydalı olan bir çox digər xüsusiyyətləri, məsələn, indiki zamanda olduğunu ehtiva edir. Qısacası, söz təsviri sözün bir çox müxtəlif xüsusiyyətlərini kodlaşdırır. Əgər biz yalnız tək bir scaled dot-product attention qatı istifadə etsəydik, bütün bu xüsusiyyətləri bir anda sorğulaya bilərdik.

Buna görə multi-head attention qatı dəyərlərin, açarların və sorğuların çoxsaylı müxtəlif xətti transformasiyalarını tətbiq edir: bu, modelə söz təsvirinin bir çox müxtəlif proyeksiyalarını müxtəlif altfəzalara tətbiq etməyə imkan verir, hər biri sözün xüsusiyyətlərinin bir alt dəstinə diqqət yetirir. Bəlkə xətti qatlardan biri söz təsvirini yalnız sözün fel olduğu məlumatının qaldığı bir altfəzaya proyeksiya edəcək, başqa bir xətti qat yalnız indiki zaman olduğunu çıxaracaq və s. Sonra scaled dot-product attention qatları axtarış mərhələsini tətbiq edir və nəhayət, biz bütün nəticələri birləşdirir və onları orijinal fəzaya geri proyeksiya edirik.

Keras `tf.keras.layers.MultiHeadAttention` qatını ehtiva edir, beləliklə transformer-in qalan hissəsini qurmaq üçün lazım olan hər şeyə sahibik. Gəlin tam encoder ilə başlayaq, bu, Şəkil 16-8-də olduğu kimidir, yalnız iki blokdan ($N = 2$) ibarət yığın istifadə edirik (altı əvəzinə), çünki böyük bir təlim setimiz yoxdur, və bir az dropout da əlavə edirik:

```
N = 2 # instead of 6
num_heads = 8
dropout_rate = 0.1
n_units = 128 # for the first dense layer in each feedforward block
```

```

encoder_pad_mask = tf.math.not_equal(encoder_input_ids, 0)[: , tf.newaxis]
Z = encoder_in

for _ in range(N):
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=Z, attention_mask=encoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))

    skip = Z
    Z = tf.keras.layers.Dense(n_units, activation="relu")(Z)
    Z = tf.keras.layers.Dense(embed_size)(Z)
    Z = tf.keras.layers.Dropout(dropout_rate)(Z)

    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))

```

Bu kod əsasən sadədir, yalnız bir şey istisna olmaqla: masking. Bu yazı yazıldığı zaman, MultiHeadAttention qatı avtomatik masking-i dəstəkləmir, buna görə onu əl ilə idarə etməliyik. Bunu necə edə bilərik?

MultiHeadAttention qatı `attention_mask` argumentini qəbul edir, bu, [batch size, max query length, max value length] formasında Boolean tensordur: hər bir sorğu ardıcılığındakı hər bir token üçün, bu maska müvafiq dəyər ardıcılığındakı hansı token-lərə diqqət yetiriləcəyini göstərir. Biz MultiHeadAttention qatına dəyərlərdəki bütün padding token-lərini nəzər almamağı söyləmək istəyirik. Beləliklə, əvvəlcə `tf.math.not_equal(encoder_input_ids, 0)` istifadə edərək padding maskasını hesablayırıq. Bu, [batch size, max sequence length] formasında Boolean tensor qaytarır. Sonra `[: , tf.newaxis]` istifadə edərək ikinci bir ox daxil edirik, [batch size, 1, max sequence length] formasında maska əldə etmək üçün. Bu, MultiHeadAttention qatını çağırarkən bu maskanı `attention_mask` kimi istifadə etməyə imkan verir: broadcasting sayəsində eyni maska hər bir sorğudakı bütün token-lər üçün istifadə ediləcək. Bu yolla, dəyərlərdəki padding token-ləri düzgün nəzərə alınmayacaq.

Lakin qat hər bir sorğu token-i üçün, o cümlədən padding token-ləri üçün çıxışlar hesablayacaq. Biz bu padding token-lərinə uyğun gələn çıxışları maskalamalıyıq. Xatırlayaq ki, Embedding qatlarında `mask_zero` istifadə etmişik və PositionalEncoding qatında `supports_masking`-i True təyin etmişik, beləliklə avtomatik maska MultiHeadAttention qatının girişlərinə (encoder_in) qədər ötürülmüşdür. Biz bundan skip connection-da faydalana bilərik: həqiqətən də, Add qatı avtomatik masking-i dəstəkləyir, buna görə Z və skip (əvvəlcə encoder_in-ə

bərabər) əlavə edildikdə, çıxışlar avtomatik olaraq düzgün maskalanır. Masking izahı koddan çox daha çox yazı tələb etdi!

İndi decoder-ə keçək! Bir daha, masking yeganə çətin hissə olacaq, buna görə onunla başlayaq. İlk multi-head attention qatı self-attention qatıdır, encoder-də olduğu kimi, lakin bu *masked* multi-head attention qatıdır, yəni səbəb (causal) dir: o, gələcəkdəki bütün token-ləri nəzər almamalıdır. Beləliklə, bizə iki maska lazımdır: padding maskası və causal maska. Gəlin onları yaradaq:

```
decoder_pad_mask = tf.math.not_equal(decoder_input_ids, 0)[: , tf.newaxis]
causal_mask = tf.linalg.band_part( # creates a lower triangular matrix

    tf.ones((batch_max_len_dec, batch_max_len_dec), tf.bool), -1, 0)
```

Padding maskası encoder üçün yaratdığımızla eynidir, lakin encoder-in girişləri əvəzinə decoder-in girişlərinə əsaslanır. Causal maska `tf.linalg.band_part()` funksiyasından istifadə edərək yaradılır, bu, bir tensor qəbul edir və diaqonal zolaqdan kənarda bütün dəyərləri sıfıra təyin edilmiş bir surətini qaytarır. Bu arqumentlərlə, `batch_max_len_dec` ölçüsündə (batch-dəki giriş ardıcılıqlarının maksimum uzunluğu) kvadrat matris alırıq, aşağı-sol üçbucaqda 1-lər və yuxarı-sağda 0-lar. Əgər bu maskanı attention maskası kimi istifadə etsək, tam olaraq istədiyimizi alırıq: birinci sorğu token-i yalnız birinci dəyər token-inə, ikincisi yalnız ilk ikisinə, üçüncüsü yalnız ilk üçünə diqqət yetirəcək və s. Başqa sözlə, sorğu token-ləri gələcəkdəki heç bir dəyər token-inə diqqət yetirə bilməz.

İndi decoder-i quraq:

```
encoder_outputs = Z # let's save the encoder's final outputs
Z = decoder_in # the decoder starts with its own inputs
for _ in range(N):
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=Z, attention_mask=causal_mask &
decoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))

    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=encoder_outputs,
attention_mask=encoder_pad_mask)
```



```
Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z, skip]))
```

```
skip = Z  
Z = tf.keras.layers.Dense(n_units, activation="relu")(Z)  
Z = tf.keras.layers.Dense(embed_size)(Z)
```

```
Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z, skip]))
```

İlk attention qatı üçün, həm padding token-lərini, həm də gələcək token-ləri maskalamaq üçün `causal_mask` & `decoder_pad_mask` istifadə edirik. Causal maska yalnız iki ölçüyə malikdir: batch ölçüsü yoxdur, lakin broadcasting onun batch-dəki bütün instanslar üzərində kopyalanmasını təmin etdiyi üçün problem deyil.

İkinci attention qatı üçün xüsusi bir şey yoxdur. Yalnız qeyd edək ki, biz `decoder_pad_mask` deyil, `encoder_pad_mask` istifadə edirik, çünki bu attention qatı dəyərləri kimi encoder-in son çıxışlarından istifadə edir.

Demək olar ki, bitdik. Yalnız son çıxış qatını əlavə etmək, modeli yaratmaq, kompilyasiya etmək və öyrətmək lazımdır:

```
Y_proba = tf.keras.layers.Dense(vocab_size, activation="softmax")(Z)  
model = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],  
                        outputs=[Y_proba])  
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",  
              metrics=["accuracy"])  
model.fit((X_train, X_train_dec), Y_train, epochs=10,  
          validation_data=((X_valid, X_valid_dec), Y_valid))
```

Təbrik edirik! Siz sıfırdan tam bir transformer qurdunuz və onu avtomatik tərcümə üçün öyrətdiniz. Bu, olduqca irəliləmiş bir səviyyədir!

İPUCU

Keras komandası yeni bir Keras NLP layihəsi yaratdı, o cümlədən transformer-i daha asan qurmaq üçün bir API. Siz həmçinin kompüter görməsi üçün yeni Keras CV layihəsi ilə də maraqlana bilərsiniz.

Lakin sahə bununla dayanmadı. İndi son irəliləyişlərin bəzilərini araşdıraq.

Transformer Modellərinin Sel Basqını

2018-ci ili "NLP üçün ImageNet anı" adlandırdılar. O vaxtdan bəri, böyük dataset-lər üzərində öyrədilmiş getdikcə daha böyük transformer əsaslı arxitekturalarla irəliləyiş heyrətamiz olmuşdur.

Əvvəlcə, Alec Radford və digər OpenAI tədqiqatçılarının GPT məqaləsi, ELMo və ULMFiT məqalələrindən əvvəl olduğu kimi, unsupervised pretraining-in effektivliyini bir daha nümayiş etdirdi, lakin bu dəfə transformer bənzəri arxitektura istifadə etməklə. Müəlliflər orijinal transformer-in decoder-ində olduğu kimi yalnız masked multi-head attention qatlarından istifadə edən 12 transformer modulundan ibarət böyük, lakin kifayət qədər sadə bir arxitekturanı öncədən öyrətdilər. Onu çox böyük bir dataset üzərində, Şekspir char-RNN-imiz üçün istifadə etdiyimiz eyni autoregressive texnika ilə öyrətdilər: sadəcə növbəti token-i proqnozlaşdırmaq. Bu, self-supervised learning formasıdır. Sonra onu müxtəlif dil tapşırıqlarında incə tənzimlədilər, hər bir tapşırıq üçün yalnız kiçik uyğunlaşmalar istifadə etdilər. Tapşırıqlar olduqca müxtəlif idi: mətn təsnifatı, *entailment* (cümlə A-nın cümlə B-ni zəruri nəticə kimi tələb edib-etmədiyi, daxil edib-etmədiyi və ya nəzərdə tutduğu), oxşarlıq (məsələn, "Nice weather today" "It is sunny" ilə çox oxşardır) və sual-cavab (mətnin bir neçə abzası kontekst verildikdə, model çox seçimli suallara cavab verməlidir).

Sonra Google-un BERT məqaləsi çıxdı: o da böyük bir korpusda self-supervised pretraining-in effektivliyini nümayiş etdirdi, GPT-yə bənzər arxitektura ilə, lakin yalnız masked olmayan multi-head attention qatları ilə, orijinal transformer-in encoder-i kimi. Bu o deməkdir ki, model təbii olaraq ikitərəflidir; buna görə BERT-də B (Bidirectional Encoder Representations from Transformers). Ən əsası, müəlliflər modelin gücünün çoxunu izah edən iki pretraining tapşırığı təklif etdilər:

Masked language model (MLM)

Bir cümlədəki hər bir sözün 15% ehtimalı maskalanır və model maskalanmış sözləri proqnozlaşdırmaq üçün öyrədilir. Məsələn, orijinal cümlə "She had fun at the birthday party" olarsa, modelə "She <mask> fun at the <mask> party" cümləsi verilə bilər və o, "had" və "birthday" sözlərini proqnozlaşdırmalıdır (digər çıxışlar nəzərə alınmayacaq). Daha dəqiq desək, hər bir seçilmiş sözün 80% ehtimalı maskalanır, 10% ehtimalı təsadüfi sözlə əvəz olunur (pretraining və fine-tuning arasındakı uyğunsuzluğu azaltmaq üçün, çünki model fine-tuning zamanı <mask> token-lərini görməyəcək) və 10% ehtimalı olduğu kimi qalır (modeli düzgün cavaba yönləndirmək üçün).

Next sentence prediction (NSP)

Model iki cümlənin ardıcıl olub-olmadığını proqnozlaşdırmaq üçün öyrədilir. Məsələn, o, "The dog sleeps" və "It snores loudly" cümlələrinin ardıcıl olduğunu, "The dog

sleeps" və "The Earth orbits the Sun" cümlələrinin isə ardıcıl olmadığını proqnozlaşdırmalıdır. Sonrakı tədqiqatlar NSP-nin əvvəlcə düşünüldüyü qədər vacib olmadığını göstərdi, buna görə daha sonrakı arxitekturaların əksəriyyətində atıldı.

Model bu iki tapşırıq üzərində eyni vaxtda öyrədilir (bax Şəkil 16-11). NSP tapşırığı üçün müəlliflər hər bir girişin əvvəlinə class token (<CLS>) daxil etdilər və müvafiq çıxış token-i modelin proqnozunu təmsil edir: cümlə B cümlə A-nı izləyir, ya da yox. İki giriş cümləsi birləşdirilir, yalnız xüsusi ayırma token-i (<SEP>) ilə ayrılır və modelə giriş kimi verilir. Modelin hər bir giriş token-inin hansı cümləyə aid olduğunu bilməsinə kömək etmək üçün, hər bir token-in mövqe embeddings-lərinin üzərinə *segment embedding* əlavə olunur: yalnız iki mümkün segment embedding var, biri cümlə A üçün, digəri cümlə B üçün. MLM tapşırığı üçün, bəzi giriş sözləri maskalanır (indi gördüyümüz kimi) və model bu sözlərin nə olduğunu proqnozlaşdırmağa çalışır. Loss yalnız NSP proqnozu və maskalanmış token-lər üzərində hesablanır, maskalanmamışlar üzərində yox.

Şəkil 16-11. BERT təlim və fine-tuning prosesi

Bu unsupervised pretraining mərhələsindən sonra, model çox böyük bir mətn korpusunda sonra müxtəlif tapşırıqlarda incə tənzimlənir, hər bir tapşırıq üçün çox az dəyişiklik edilir. Məsələn, sentiment analysis kimi mətn təsnifatı üçün, class token-ə uyğun gələn birinci istisna olmaqla bütün çıxış token-ləri nəzərə alınmır və yeni bir çıxış qatı əvvəlkinə əvəz edir, o, sadəcə NSP üçün ikili təsnifat qatı idi.

2019-cu ilin fevralında, BERT nəşr olunduqdan cəmi bir neçə ay sonra, Alec Radford, Jeffrey Wu və digər OpenAI tədqiqatçıları GPT-2 məqaləsini nəşr etdilər, bu, GPT-yə çox bənzər, lakin daha böyük (1.5 milyarddan çox parametrlə!) arxitektura təklif etdi. Tədqiqatçılar yeni və təkmilləşdirilmiş GPT modelinin *zero-shot learning* (ZSL) həyata keçirə bildiyini, yəni heç bir fine-tuning olmadan bir çox tapşırıqda yaxşı performans əldə edə bildiyini göstərdilər. Bu, getdikcə daha böyük modellərə doğru yarışın başlanğıcı idi: Google-un Switch Transformers (2021-ci ilin yanvarında təqdim edilmişdir) 1 trilyon parametr istifadə etdi və tezliklə daha böyük modellər ortaya çıxdı, məsələn, Pekin Süni İntellekt Akademiyasının (BAI) Wu Dao 2.0 modeli 2021-ci ilin iyununda elan edildi.

Nəhəng modellərə doğru bu tendensiyanın xoşagəlməz nəticəsi, yalnız yaxşı maliyyələşdirilmiş təşkilatların belə modelləri öyrətməyi ödəyə bilməsidir: bu, asanlıqla yüz minlərlə dollar və ya daha çox xərcləyə bilər. Və tək bir modeli öyrətmək üçün tələb olunan enerji, bir Amerika ev təsərrüfatının bir neçə il ərzindəki elektrik istehlakına bərabərdir; bu, heç də ekoloji cəhətdən dost deyil. Bu modellərin bir çoxu müntəzəm aparatda istifadə etmək üçün çox böyükdür: RAM-a sığmaz və dəhşətli dərəcədə yavaş olardı. Nəhayət, bəziləri o qədər bahadır ki, ictimaiyyətə açıqlanmır.

Xoşbəxtlikdən, ixtiraçı tədqiqatçılar transformer-ləri kiçiltmək və onları daha məlumat-səmərəli etmək üçün yeni yollar tapırlar. Məsələn, Hugging Face-dən Victor Sanh və digərləri tərəfindən 2019-cu ilin oktyabrında təqdim edilən DistilBERT modeli, BERT əsaslı kiçik və sürətli bir transformer modelidir. O, Hugging Face-in əla model hub-da, minlərlə başqaları ilə birlikdə mövcuddur – bu fəslin sonrakı hissəsində bir nümunə görəcəksiniz.

DistilBERT *distillation* (buna görə adı) istifadə edərək öyrədilmişdir: bu, müəllim modelindən tələbə modelinə bilik ötürmək deməkdir, tələbə modeli adətən müəllim modelindən çox kiçikdir. Bu, adətən müəllimin hər bir təlim instansı üçün proqnozlaşdırılan ehtimallarını tələbə üçün hədəf kimi istifadə etməklə edilir. Təəccüblüdür ki, distillasiya çox vaxt tələbəni müəllim ilə eyni dataset-də sıfırdan öyrətməkdən daha yaxşı işləyir! Həqiqətən, tələbə müəllimin daha incə etiketlərindən faydalanır.

BERT-dən sonra daha çox transformer arxitekturaları, demək olar ki, hər ay çıxdı, çox vaxt bütün NLP tapşırıqlarında ən müasir vəziyyəti yaxşılaşdırdı: XLNet (iyun 2019), RoBERTa (iyul 2019), StructBERT (avqust 2019), ALBERT (sentyabr 2019), T5 (oktyabr 2019), ELECTRA (mart 2020), GPT3 (may 2020), DeBERTa (iyun 2020), Switch Transformers (yanvar 2021), Wu Dao 2.0 (iyun 2021), Gopher (dekabr 2021), GPT-NeoX-20B (fevral 2022), Chinchilla (mart 2022), OPT (may 2022) və siyahı davam edir. Bu modellərin hər biri yeni fikirlər və texnikalar gətirdi, lakin mən xüsusilə Google tədqiqatçılarının T5 məqaləsini bəyənim: o, bütün NLP tapşırıqlarını text-to-text kimi qələmə verir, encoder-decoder transformer istifadə edərək. Məsələn, "I like soccer"-u ispan dilinə tərcümə etmək üçün, modelə "translate English to Spanish: I like soccer" giriş cümləsi ilə zəng edə bilərsiniz və o, "me gusta el fútbol" çıxarır. Bir abzası xülasə etmək üçün, sadəcə "summarize:" ilə başlayan abzası daxil edirsiniz və o, xülasəni çıxarır. Təsnifat üçün, yalnız prefiksi "classify:" olaraq dəyişdirməlisiniz və model sinif adını mətn olaraq çıxarır. Bu, modeli istifadə etməyi sadələşdirir, həmçinin onu daha çox tapşırıqda pretraining etməyə imkan verir.

Sonuncu, lakin ən az deyil, 2022-ci ilin aprelində, Google tədqiqatçıları *Pathways* adlı yeni böyük miqyaslı təlim platformasından (Fəsil 19-da qısaca müzakirə edəcəyik) istifadə edərək *Pathways Language Model* (PaLM) adlı nəhəng bir dil modelini öyrətdilər, 540 milyard parametr ilə, 6,000-dən çox TPU istifadə edərək. İnanılmaz ölçüsündən başqa, bu model standart transformer-dir, yalnız decoder-lər (yəni masked multi-head attention qatları) ilə, bir neçə kiçik düzəlişlərlə (təfərrüatlar üçün məqaləyə baxın). Bu model bütün növ NLP tapşırıqlarında, xüsusən təbii dil anlama (NLU) sahəsində inanılmaz performans əldə etdi. O, zarafatları izah etmək, suallara ətraflı addım-addım cavablar vermək və hətta kod yazmaq kimi təsirli feat-lara qadirdir. Bu, qismən modelin ölçüsü, həm də bir neçə ay əvvəl başqa bir Google tədqiqatçıları komandası tərəfindən təqdim edilən *Chain of thought prompting* texnikası sayəsindədir.

Sual-cavab tapşırıqlarında, adi prompting adətən bir neçə sual və cavab nümunəsini ehtiva edir, məsələn: "Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: 11." Prompt daha sonra faktiki sual ilə davam edir, məsələn "Q: John takes care of 10 dogs. Each dog takes .5 hours a day to walk and take care of their business. How many hours a week does he spend taking care of dogs? A:", və modelin işi cavabı əlavə etməkdir: bu halda "35."

Lakin chain of thought prompting ilə, nümunə cavablar nəticəyə gətirən bütün əsaslandırma addımlarını ehtiva edir. Məsələn, "A: 11" əvəzinə prompt "A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$." ehtiva edir. Bu, modeli faktiki suala ətraflı cavab verməyə təşviq edir, məsələn "John takes care of 10 dogs. Each dog takes .5 hours a day to walk and take care of their business. So that is $10 \times .5 = 5$ hours a day. 5 hours a day $\times 7$ days a week = 35 hours a week. The answer is 35 hours a week." Bu, məqalədən faktiki bir nümunədir!

Model nəinki adi prompting istifadə etməkdən daha tez-tez düzgün cavab verir – biz modeli hər şeyi düşünməyə təşviq edirik – həm də bütün əsaslandırma addımlarını təqdim edir, bu, modelin cavabının arxasındakı məntiqi daha yaxşı anlamaq üçün faydalı ola bilər.

Transformers NLP-ni ələ keçirdi, lakin bununla dayanmadı: tezliklə kompüter görməsinə də genişləndi.

Vision Transformers

Attention mexanizmlərinin NMT-dən kənarda ilk tətbiqlərindən biri vizual attention istifadə edərək şəkil başlıqları yaratmaq idi: bir convolutional neural network əvvəlcə şəkli emal edir və bəzi feature map-lar çıxarır, sonra attention mexanizmi ilə təchiz edilmiş bir decoder RNN başlığı bir-bir söz yaradır.

Hər bir decoder zaman addımında (yəni hər bir söz), decoder şəklində düzgün hissəsinə diqqət yetirmək üçün attention modelindən istifadə edir. Məsələn, Şəkil 16-12-də model "A woman is throwing a frisbee in a park" başlığını yaratdı və "frisbee" sözünü çıxarmaq üzrə olduğu zaman decoder-in giriş şəklində hansı hissəsinə diqqət yetirdiyini görə bilərsiniz: aydındır ki, diqqətinin çox hissəsi frisbee-yə yönəlmişdi.

Şəkil 16-12. Vizual attention: giriş şəkli (sol) və "frisbee" sözünü istehsal etməzdən əvvəl modelin diqqət mərkəzi (sağ)

İZAHLIQLIQ (EXPLAINABILITY)

Attention mexanizmlərinin bir əlavə üstünlüyü, modeli nəyin çıxışını istehsal etməyə yönəltdiyini anlamağı asanlaşdırmasıdır. Buna *explainability* deyilir. Bu, model səhv etdikdə xüsusilə faydalı ola bilər: məsələn, qarda gəzən bir itin şəkli "a wolf walking in the snow" olaraq etiketlənirsə, "wolf" sözünü çıxararkən modelin nəyə diqqət yetirdiyini yoxlaya bilərsiniz. Siz onun yalnız itə deyil, həm də qara diqqət yetirdiyini görə bilərsiniz, bu da mümkün bir izahı işarə edir: bəlkə modelin itləri canavarlardan ayırmağı öyrənmə üsulu ətrafda çox qar olub-olmamasını yoxlamaqdır. Sonra modeli qarsız canavarlar və qarlı itlərlə daha çox şəkillə öyrədərək bunu düzəldə bilərsiniz. Bu nümunə, izahlıq üçün fərqli bir yanaşma – təsnifatçının proqnozu ətrafında yerli olaraq yozula bilən bir model öyrənmək – istifadə edən Marco Tulio Ribeiro və digərlərinin 2016-cı il əla məqaləsindən gəlir.

Bəzi tətbiqlərdə izahlıq yalnız modeli debug etmək üçün bir alət deyil, qanuni tələb ola bilər – sizə kredit verib-verməyəcəyinə qərar verən bir sistemi düşünün.

Transformers 2017-ci ildə ortaya çıxdıqda və insanlar onları NLP-dən kənarda sınaqdan keçirməyə başladıda, əvvəlcə CNN-lərlə birlikdə, onları əvəz etmədən istifadə edildi. Bunun əvəzinə, transformers ümumiyyətlə RNN-ləri əvəz etmək üçün istifadə edildi, məsələn, şəkil başlığı modellərində. Transformers 2020-ci ildə Facebook tədqiqatçılarının məqaləsində bir az daha vizual oldu, bu, obyekt aşkarlama üçün hibrid CNN–transformer arxitekturası təklif etdi. Yenə CNN əvvəlcə giriş şəkillərini emal edir və bir sıra feature map-lar çıxarır, sonra bu feature map-lar ardıcılıqlara çevrilir və transformer-a qidalandırılır, o, bounding box proqnozlarını çıxarır. Lakin yenə də vizual işin çoxu hələ CNN tərəfindən edilir.

Sonra, 2020-ci ilin oktyabrında, Google tədqiqatçıları komandası tam transformer əsaslı vizual model təqdim edən bir məqalə nəşr etdi, *vision transformer* (ViT) adlanır. Fikir təəccüblü dərəcədə sadədir: şəkli kiçik 16×16 kvadratlara bölmək və kvadratların ardıcılığını söz təsvirlərinin ardıcılığı kimi qəbul etmək. Daha dəqiq desək, kvadratlar əvvəlcə $16 \times 16 \times 3 = 768$ -ölçülü vektorlara düzləşdirilir – 3 RGB rəng kanalları üçündür – sonra bu vektorlar xətti qatdan keçir, onları transformasiya edir, lakin onların ölçülülüyünü saxlayır. Yaranan vektorlar ardıcılığı daha sonra söz embeddings-ləri ardıcılığı kimi qəbul edilə bilər: bu, mövqe embeddings-ləri əlavə etmək və nəticəni transformer-a ötürmək deməkdir. Hamısı budur! Bu model ImageNet şəkil təsnifatında ən müasir vəziyyəti üstələdi, lakin ədalətli olmaq üçün müəlliflər təlim üçün 300 milyondan çox əlavə şəkil istifadə etməli oldular. Bu, mənə kəsb edir, çünki transformer-lər convolution şəbəkələri qədər *inductive biases* malik deyillər, buna görə CNN-lərin dolayısı ilə qəbul etdiyi şeyləri öyrənmək üçün əlavə məlumat lazımdır.

QEYD

İnduktiv qərəz (inductive bias) modelin arxitekturasına görə etdiyi dolayı fərziyyədir. Məsələn, xətti modellər məlumatların, yaxşı, xətti olduğunu dolayısi ilə fərz edir. CNN-lər bir yerdə öyrənilən naxışların digər yerlərdə də faydalı olacağını dolayısi ilə fərz edir. RNN-lər girişlərin sıralı olduğunu və son token-lərin daha köhnə token-lərdən daha vacib olduğunu dolayısi ilə fərz edir. Modelin nə qədər çox induktiv qərəzləri varsa, onların düzgün olduğunu fərz etsək, model bir o qədər az təlim məlumatı tələb edəcək. Lakin dolayı fərziyyələr səhvdirsə, model böyük bir dataset-də öyrədilsə belə, zəif performans göstərə bilər.

Cəmi iki ay sonra, Facebook tədqiqatçıları komandası *data-efficient image transformers* (DeiT) təqdim edən bir məqalə nəşr etdi. Onların modeli heç bir əlavə təlim məlumatı tələb etmədən ImageNet-də rəqabətli nəticələr əldə etdi. Modelin arxitekturası orijinal ViT ilə demək olar ki, eynidir, lakin müəlliflər ən müasir CNN modellərindən öz modellərinə bilik ötürmək üçün distillasiya texnikasından istifadə etdilər.

Sonra, 2021-ci ilin martında, DeepMind *Perceiver* arxitekturasını təqdim edən mühüm bir məqalə nəşr etdi. Bu, *multimodal* transformer-dır, yəni ona mətn, şəkillər, audio və ya demək olar ki, istənilən başqa modalite qidalandıra bilərsiniz. O vaxta qədər transformer-lər attention qatlarındakı performans və RAM darboğazı səbəbindən kifayət qədər qısa ardıcılıqlarla məhdudlaşmışdı. Bu, audio və ya video kimi modalitələri istisna edir və tədqiqatçıları şəkilləri piksel ardıcılığı əvəzinə yamaq ardıcılığı kimi qəbul etməyə məcbur edirdi. Darboğaz self-attention-dan qaynaqlanır, burada hər bir token hər digər token-ə diqqət yetirməlidir: əgər giriş ardıcılığının M token-i varsa, attention qatı $M \times M$ matrisini hesablamalıdır, bu, M çox böyükdürsə nəhəng ola bilər.

Perceiver, girişlərin kifayət qədər qısa *latent representation* (gizli təsvir) – N token-dən ibarət, adətən cəmi bir neçə yüz – tədricən yaxşılaşdırmaqla bu problemi həll edir. (*Latent* sözü gizli və ya daxili deməkdir.) Model yalnız cross-attention qatlarından istifadə edir, onları latent təsviri sorğular, (bəlkə də böyük) girişləri isə dəyərlər kimi qidalandırır. Bu, yalnız $M \times N$ matrisinin hesablanması tələb edir, beləliklə hesablama mürəkkəbliyi M ilə xətti olur, kvadratik əvəzinə. Bir neçə cross-attention qatından keçdikdən sonra, hər şey yaxşı gedərsə, latent təsvir girişlərdə vacib olan hər şeyi tutur. Müəlliflər həmçinin ardıcıl cross-attention qatları arasında çəkirlərin paylaşılmasını təklif etdilər: əgər bunu etsəniz, Perceiver effektiv olaraq bir RNN-ə çevrilir. Həqiqətən də, paylaşılan cross-attention qatları müxtəlif zaman addımlarında eyni yaddaş hüceyrəsi kimi görünə bilər və latent təsvir hüceyrənin kontekst vektoruna uyğun gəlir. Eyni girişlər hər zaman addımında yaddaş hüceyrəsinə təkrar qidalandırılır. Görünür, RNN-lər hələ də ölü deyil!

Cəmi bir ay sonra, Mathilde Caron və digərləri DINO-nu təqdim etdilər, tamamilə etiketlərsiz, self-supervision istifadə edərək öyrədilən və yüksək dəqiqlikli semantic segmentation qabiliyyətinə malik təsirli bir vision transformer. Təlim zamanı model ikiqatlaşdırılır, bir şəbəkə müəllim, digəri tələbə kimi çıxış edir. Gradient descent yalnız tələbəyə təsir edir, müəllimin çəkilişi isə tələbənin çəkilişinin eksponensial hərəkətli ortalamasıdır. Tələbə müəllimin proqnozlarına uyğunlaşmaq üçün öyrədilir: onlar demək olar ki, eyni model olduğundan, buna *self-distillation* deyilir. Hər bir təlim addımında, giriş şəkilləri müəllim və tələbə üçün fərqli üsullarla artırılır, buna görə eyni şəkli görmürlər, lakin onların proqnozları uyğun olmalıdır. Bu, onları yüksək səviyyəli təsvirlər tapmağa məcbur edir. Həm tələbə, həm də müəllimin girişləri tamamilə nəzər almadan hər zaman eyni şeyi çıxaracağı *mode collapse* un qarşısını almaq üçün, DINO müəllimin çıxışlarının hərəkətli ortalamasını izləyir və müəllimin proqnozlarını onların orta hesabla sıfır ətrafında qalmasını təmin edəcək şəkildə tənzimləyir. DINO həmçinin müəllimi proqnozlarında yüksək inamı olmağa məcbur edir: buna *sharpening* deyilir. Birlikdə, bu texnikalar müəllimin çıxışlarında müxtəlifliyi qoruyur.

2021-ci il məqaləsində, Google tədqiqatçıları məlumatların miqdarından asılı olaraq ViT-ləri necə böyütmək və ya kiçiltməyi göstərdilər. Onlar ImageNet-də 90.4%-dən çox top-1 dəqiqliyinə çatan nəhəng 2 milyard parametrlili bir model yaratmağı bacardılar. Əksinə, onlar yalnız 10,000 şəkil (hər sinif üçün cəmi 10 şəkil) istifadə edərək ImageNet-də 84.8%-dən çox top-1 dəqiqliyinə çatan kiçilmiş bir model də öyrətdilər.

Və vizual transformer-lərdəki irəliləyiş bu günə qədər davamlı olaraq davam etdi. Məsələn, 2022-ci ilin martında Mitchell Wortsman və digərlərinin məqaləsi, əvvəlcə bir neçə transformer-i öyrətməyin, sonra yeni və təkmilləşdirilmiş bir model yaratmaq üçün onların çəkilişini ortalamayın mümkün olduğunu nümayiş etdirdi. Bu, ansambl (bax Fəsil 7) bənzəyir, yalnız sonunda tək bir model var, yəni çıxarma vaxtı cəzası yoxdur.

Transformers-dəki ən son tendensiya, çox vaxt zero-shot və ya few-shot learning qabiliyyətinə malik böyük multimodal modellər qurmaqdır. Məsələn, OpenAI-nin 2021-ci il CLIP məqaləsi, başlıqları şəkillərlə uyğunlaşdırmaq üçün öncədən öyrədilmiş böyük bir transformer modeli təklif etdi: bu tapşırıq ona əla şəkil təsvirləri öyrənməyə imkan verir və model daha sonra "a photo of a cat" kimi sadə mətn promptları istifadə edərək şəkil təsnifatı kimi tapşırıqlar üçün birbaşa istifadə edilə bilər. Qısa müddət sonra OpenAI, mətn promptları əsasında heyranəmiz şəkillər yarada bilən DALL·E-ni elan etdi. DALL·E 2, diffusion modeli (bax Fəsil 17) istifadə edərək daha yüksək keyfiyyətli şəkillər yaradır.

2022-ci ilin aprelində, DeepMind Flamingo məqaləsini nəşr etdi, o, mətn, şəkillər və videolar daxil olmaqla çoxsaylı modalitələr üzrə geniş bir tapşırıq spektrində öncədən öyrədilmiş modellər ailəsini təqdim etdi. Tək bir model sual-cavab, şəkil başlıqları və

daha çoxu kimi çox fərqli tapşırıqlarda istifadə edilə bilər. Qısa müddət sonra, 2022-ci ilin mayında, DeepMind GATO-nu təqdim etdi, bir reinforcement learning agent (RL Fəsil 18-də təqdim ediləcək) üçün siyasət kimi istifadə edilə bilən multimodal model. Eyni transformer sizinlə söhbət edə, şəkillərə başlıq yazsa, Atari oyunları oynaya, (simulyasiya edilmiş) robot qollarını idarə edə və daha çoxunu edə bilər, hamısı "yalnız" 1.2 milyard parametrlə. Və macərə davam edir!

QEYD

Bu heyramiz irəliləyişlər bəzi tədqiqatçıların insan səviyyəsində AI-nin yaxın olduğunu, "scale is all you need" (miqyas hər şeydir) olduğunu və bu modellərin bəzilərinin "bir az şüurlu" ola biləcəyini iddia etməsinə səbəb oldu. Digərləri isə heyramiz irəliləyişə baxmayaraq, bu modellərin hələ də insan zəkasının etibarlılığına və uyğunlaşma qabiliyyətinə, simvolik olaraq düşünmək, tək bir nümunəyə əsaslanaraq ümumiləşdirmək qabiliyyətimizə və daha çoxuna malik olmadığını qeyd edirlər.

Gördüyünüz kimi, transformers hər yerdədir! Və xoş xəbər odur ki, ümumiyyətlə transformer-ləri özünüz tətbiq etmək məcburiyyətində deyilsiniz, çünki bir çox əlavədən öyrədilmiş modellər TensorFlow Hub və ya Hugging Face-in model hub-dan yükləmək üçün asanlıqla mövcuddur. TF Hub-dan bir modeli necə istifadə edəcəyinizi artıq görmüsünüz, buna görə bu fəslə Hugging Face-in ekosisteminə qısa bir baxış ilə bağlayaq.

Hugging Face-in Transformers Kitabxanası

Bu gün transformers haqqında Hugging Face-i qeyd etmədən danışmaq mümkün deyil, bu, NLP, vizyon və daha çox sahələr üçün asan istifadə olunan açıq qaynaq alətlər ekosistemini quran bir AI şirkətidir. Onların ekosisteminin mərkəzi komponenti Transformers kitabxanasıdır, bu, əvvəlcədən öyrədilmiş modeli, o cümlədən müvafiq tokenizer-i asanlıqla yükləməyə və sonra lazım olduqda öz dataset-inizdə incə tənzimləməyə imkan verir. Üstəlik, kitabxana TensorFlow, PyTorch və JAX (Flax kitabxanası ilə) dəstəkləyir.

Transformers kitabxanasından istifadə etməyin ən sadə yolu `transformers.pipeline()` funksiyasından istifadə etməkdir: sadəcə sentiment analysis kimi hansı tapşırığı istədiyinizi təyin edirsiniz və o, istifadəyə hazır bir default əvvəlcədən öyrədilmiş modeli yükləyir – həqiqətən daha sadə ola bilməz:

```
from transformers import pipeline
```

```
classifier = pipeline("sentiment-analysis") # many other tasks are available
```

```
result = classifier("The actors were very convincing.")
```

Nəticə, hər bir giriş mətni üçün bir lüğət ehtiva edən Python siyahısıdır:

```
>>> result
```

```
[{'label': 'POSITIVE', 'score': 0.9998071789741516}]
```

Bu nümunədə, model cümlənin müsbət olduğunu təxminən 99.98% inamla düzgün tapdı. Əlbəttə, modelə bir batch cümlə də ötürə bilərsiniz:

```
>>> classifier(["I am from India.", "I am from Iraq."])
```

```
[{'label': 'POSITIVE', 'score': 0.9896161556243896},
```

```
 {'label': 'NEGATIVE', 'score': 0.9811071157455444}]
```

BIAS VƏ ƏDALƏTLİLİK (FAIRNESS)

Çıxışın göstərdiyi kimi, bu xüsusi təsnifatçı hindliləri sevir, lakin iraqilərə qarşı ciddi qərəzlidir. Bu kodu öz ölkəniz və ya şəhərinizlə sınağa bilərsiniz. Belə bir arzu olunmaz qərəz ümumiyyətlə böyük ölçüdə təlim məlumatlarının özündən qaynaqlanır: bu halda, təlim məlumatlarında İraqdakı müharibələrlə əlaqəli çoxsaylı mənfi cümlələr var idi. Bu qərəz daha sonra fine-tuning prosesi zamanı gücləndirildi, çünki model yalnız iki sinif arasında seçim etməyə məcbur edildi: müsbət və ya mənfi. Fine-tuning zamanı neytral sinif əlavə etsəniz, ölkə qərəzi əsasən yox olur. Lakin təlim məlumatları qərəzin yeganə mənbəyi deyil: modelin arxitekturası, təlim üçün istifadə olunan loss və ya regularizasiya növü, optimizator; bunların hamısı modelin nə öyrəndiyinə təsir edə bilər. Hətta əsasən qərəzsiz bir model də qərəzli şəkildə *istifadə* edilə bilər, sorğu suallarının qərəzli ola biləcəyi kimi.

Al-da qərəzi anlamaq və onun mənfi təsirlərini azaltmaq hələ də aktiv tədqiqat sahəsidir, lakin bir şey dəqiqdir: modeli istehsalata tələsik yerləşdirməzdən əvvəl dayanıb düşünməlisiniz. Modelin həтта dolayısı ilə necə zərər verə biləcəyini özünüə soruşun. Məsələn, modelin proqnozları kiməsə kredit verib-verməməyə qərar vermək üçün istifadə olunursa, proses ədalətli olmalıdır. Beləliklə, modelin performansını təkcə bütün test seti üzərində orta hesabla deyil, həm də müxtəlif alt qruplar üzərində qiymətləndirdiyinizə əmin olun: məsələn, modelin orta hesabla çox yaxşı işlədiyini, lakin bəzi insan kateqoriyaları üçün performansının acınacaqlı olduğunu görə

bilərsiniz. Siz həmçinin kontrafaktual testlər keçirmək istəyə bilərsiniz: məsələn, modelin proqnozlarının birinin cinsini dəyişdirdiyiniz zaman dəyişmədiyini yoxlamaq istəyə bilərsiniz.

Əgər model orta hesabla yaxşı işləyirsə, onu istehsala itələmək və başqa bir şeyə keçmək cazibədardır, xüsusən də daha böyük bir sistemin yalnız bir komponentidirsə. Lakin ümumiyyətlə, bu cür məsələləri düzəltməsəniz, başqa heç kim düzəltməyəcək və modeliniz yaxşıdan daha çox zərər verə bilər. Həll problemdən asılıdır: dataset-i balanslaşdırmaq, fərqli bir dataset-də fine-tuning etmək, başqa bir əvvəlcədən öyrədilmiş modelə keçmək, modelin arxitekturasını və ya hiperparametrlərini tənzimləmək və s. tələb edə bilər.

`pipeline()` funksiyası verilmiş tapşırıq üçün defolt modeldən istifadə edir. Məsələn, sentiment analysis kimi mətn təsnifatı tapşırıqları üçün, bu yazı yazıldığı vaxtda, o, `distilbert-base-uncased-finetuned-sst-2-english`-ə – ingilis Wikipedia və ingilis kitabları korpusunda öyrədilmiş və Stanford Sentiment Treebank v2 (SST 2) tapşırığında incə tənzimlənmiş uncased tokenizer-i olan DistilBERT modelinə defolt olur. Fərqli bir model əl ilə təyin etmək də mümkündür. Məsələn, Multi-Genre Natural Language Inference (MultiNLI) tapşırığında incə tənzimlənmiş bir DistilBERT modelindən istifadə edə bilərsiniz, o, iki cümləni üç sinfə təsnif edir: ziddiyyət, neytral və ya entailment. Bunu necə etmək olar:

```
>>> model_name = "huggingface/distilbert-base-uncased-finetuned-mnli"
>>> classifier_mnli = pipeline("text-classification", model=model_name)
>>> classifier_mnli("She loves me. [SEP] She loves me not.")

[{'label': 'contradiction', 'score': 0.9790192246437073}]
```

İPUCU

Mövcud modelləri <https://huggingface.co/models> , tapşırıqların siyahısını isə <https://huggingface.co/tasks> ünvanında tapa bilərsiniz.

`pipeline` API-si çox sadə və rahatdır, lakin bəzən daha çox nəzarətə ehtiyacınız olacaq. Belə hallar üçün Transformers kitabxanası bir çox siniflər, o cümlədən hər cür tokenizer-lər, modellər, konfigurasiyalar, callback-lər və daha çoxunu təqdim edir. Məsələn, eyni DistilBERT modelini, onun müvafiq tokenizer-i ilə birlikdə, `TFAutoModelForSequenceClassification` və `AutoTokenizer` siniflərindən istifadə edərək yükləyək:

```
from transformers import TFAutoModelForSequenceClassification, AutoTokenizer,
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = TFAutoModelForSequenceClassification.from_pretrained(model_name)
```

Sonra, bir neçə cümlə cütünü tokenize edək. Bu kodda, padding-i aktivləşdiririk və TensorFlow tensorları əvəzinə Python siyahılarını istədiyimizi təyin edirik:

```
token_ids = tokenizer(["I like soccer. [SEP] We all love soccer!",  
                      "Joe lived for a very long time. [SEP] Joe is  
old."],
```

```
padding=True, return_tensors="tf")
```

İPUCU

Tokenizer-a "Sentence 1 [SEP] Sentence 2" ötürmək əvəzinə, ona bir tuple ötürə bilərsiniz: ("Sentence 1", "Sentence 2").

Çıxış, token ID-lərinin ardıcılığını, həmçinin padding token-ləri üçün 0-ları ehtiva edən maskanı ehtiva edən BatchEncoding sinfinin lüğət bənzəri nümunəsidir:

```
>>> token_ids  
{'input_ids': <tf.Tensor: shape=(2, 15), dtype=int32, numpy=  
array([[ 101, 1045, 2066, 4715, 1012, 102, 2057, 2035, 2293, 4715, 999,  
102, 0, 0, 0],  
       [ 101, 3533, 2973, 2005, 1037, 2200, 2146, 2051, 1012, 102, 3533,  
2003, 2214, 1012, 102]]), dtype=int32>,  
'attention_mask': <tf.Tensor: shape=(2, 15), dtype=int32, numpy=  
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0],  
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]), dtype=int32>}
```

Tokenizer-ı çağırarkən `return_token_type_ids=True` təyin etsəniz, hər bir token-in hansı cümləyə aid olduğunu göstərən əlavə bir tensor da alacaqsınız. Bu, bəzi modellər tərəfindən tələb olunur, lakin DistilBERT tərəfindən deyil.

Sonra, bu BatchEncoding obyektini birbaşa modelə ötürə bilərik; o, proqnozlaşdırılan sinif logits-lərini ehtiva edən bir TFSequenceClassifierOutput obyektini qaytarır:

```
>>> outputs = model(token_ids)  
>>> outputs  
TFSequenceClassifierOutput(loss=None, logits=[<tf.Tensor: [...] numpy=  
array([[ -2.1123817 ,  1.1786783 ,  1.4101017 ]],
```

```
[ -0.01478387,  1.0962474 , -0.9919954 ], dtype=float32)>], [...])
```

Nəhayət, bu logits-ləri sinif ehtimallarına çevirmək üçün softmax aktivasiya funksiyasını tətbiq edə və hər bir giriş cümlə cütü üçün ən yüksək ehtimallı sinfi proqnozlaşdırmaq üçün `argmax()` funksiyasından istifadə edə bilərik:

```
>>> Y_probas = tf.keras.activations.softmax(outputs.logits)
>>> Y_probas
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[0.01619702, 0.43523544, 0.5485676 ],
       [0.08672056, 0.85204804, 0.06123142]], dtype=float32)>
>>> Y_pred = tf.argmax(Y_probas, axis=1)
>>> Y_pred # 0 = contradiction, 1 = entailment, 2 = neutral

<tf.Tensor: shape=(2,), dtype=int64, numpy=array([2, 1])>
```

Bu nümunədə, model birinci cümlə cütünü neytral olaraq (mənim futbolu sevməyim hamının sevdiyi demək deyil) və ikinci cütü entailment (Joe həqiqətən olduqca yaşlı olmalıdır) olaraq düzgün təsnif edir.

Əgər bu modeli öz dataset-inizdə incə tənzimləmək istəyirsinizsə, modeli Keras ilə adi qaydada öyrədə bilərsiniz, çünki o, bir neçə əlavə metoda malik adi Keras modelidir. Lakin model logits çıxardığı üçün, "sparse_categorical_crossentropy" əvəzinə `tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)` loss-dan istifadə etməlisiniz. Üstəlik, model təlim zamanı BatchEncoding girişlərini dəstəkləmir, buna görə onun `data` atributundan istifadə edərək adi bir lüğət əldə etməlisiniz:

```
sentences = [("Sky is blue", "Sky is red"), ("I love her", "She loves me")]
X_train = tokenizer(sentences, padding=True, return_tensors="tf").data
y_train = tf.constant([0, 2]) # contradiction, neutral
loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(loss=loss, optimizer="nadam", metrics=["accuracy"])

history = model.fit(X_train, y_train, epochs=2)
```

Hugging Face həmçinin standart bir dataset-i (məsələn, IMDb) və ya xüsusi bir dataset-i asanlıqla yükləmək və modelinizi incə tənzimləmək üçün istifadə etmək üçün Datasets kitabxanası qurmuşdur. Bu, TensorFlow Datasets-ə bənzəyir, lakin həmçinin masking kimi ümumi preprocessing tapşırıqlarını yerində yerinə yetirmək üçün alətlər təqdim edir. Dataset-lərin siyahısı <https://huggingface.co/datasets> ünvanında mövcuddur.

Bu, Hugging Face-in ekosistemi ilə başlamaq üçün kifayət etməlidir. Daha çox öyrənmək üçün <https://huggingface.co/docs> səhifəsinə baş çəkə bilərsiniz, burada bir çox tutorial notebook-lar, videolar, tam API və daha çoxu var. Mən həmçinin O'Reilly-nin *Natural Language Processing with Transformers: Building Language Applications with Hugging Face* kitabını (Lewis Tunstall, Leandro von Werra və Thomas Wolf – hamısı Hugging Face komandasından) tövsiyə edirəm.

Növbəti fəsildə autoencoders istifadə edərək nəzarətsiz şəkildə dərin təsvirləri necə öyrənəcəyimizi müzakirə edəcəyik və generative adversarial networks istifadə edərək şəkillər və daha çoxunu istehsal edəcəyik!

Məşqlər

1. Stateful RNN ilə stateless RNN arasında üstünlüklər və çatışmazlıqlar hansılardır?
2. İnsanlar avtomatik tərcümə üçün sadə seq2seq RNN-lər əvəzinə niyə encoder–decoder RNN-lərindən istifadə edirlər?
3. Dəyişən uzunluqlu giriş ardıcılıqları ilə necə məşğul olmaq olar? Bəs dəyişən uzunluqlu çıxış ardıcılıqları?
4. Beam search nədir və niyə ondan istifadə edərdiniz? Onu tətbiq etmək üçün hansı alətdən istifadə edə bilərsiniz?
5. Attention mexanizmi nədir? O, necə kömək edir?
6. Transformer arxitekturasında ən vacib qat hansıdır? Onun məqsədi nədir?
7. Nə vaxt sampled softmax istifadə etmək lazımdır?
8. *Embedded Reber grammars* Hochreiter və Schmidhuber tərəfindən LSTM-lər haqqında məqalələrində istifadə edilmişdir. Bunlar "BPBTSXXVPSEPE" kimi sətirlər istehsal edən süni qrammatikalardır. Jenny Orr-un bu mövzuya yaxşı girişinə baxın, sonra müəyyən bir embedded Reber qrammatikasını seçin (məsələn, Orr-un səhifəsində təsvir olunan), sonra bir RNN-i sətirin həmin qrammatikaya riayət edib-etmədiyini müəyyən etmək üçün öyrədin. Əvvəlcə təxminən 50% qrammatikaya riayət edən və 50% riayət etməyən sətirləri ehtiva edən bir təlim batch-i yarada bilən bir funksiya yazmalısınız.
9. Bir tarix sətirini bir formatdan digərinə çevirə bilən encoder–decoder modeli öyrədin (məsələn, "April 22, 2019" dan "2019-04-22" yə).
10. Keras saytındakı "Natural language image search with a Dual Encoder" nümunəsini nəzərdən keçirin. Siz həm şəkilləri, həm də mətni eyni embedding fəzasında təmsil edə bilən bir modelin necə qurulacağını öyrənəcəksiniz. Bu, OpenAI-nin CLIP modelində olduğu kimi, mətn promptundan istifadə edərək şəkilləri axtarmağa imkan verir.
11. Hugging Face Transformers kitabxanasından istifadə edərək mətn yarada bilən əvvəlcədən öyrədilmiş bir dil modelini (məsələn, GPT) yükləyin və daha

inandırıcı Şekspir mətni yaratmağa çalışın. Modelin `generate()` metodundan istifadə etməlisiniz – daha ətraflı məlumat üçün Hugging Face sənədlərinə baxın.

Bu məşqlərin həlləri bu fəslin notebook-unun sonunda, <https://homl.info/colab3> ünvanında mövcuddur.